ARM7TDMI®

Revision: Rev3a and r3p0

Integration Manual

Confidential

ARM®

Copyright © 2005 ARM Limited. All rights reserved.
ARM DII 0101A

ARM7TDMI
Integration Manual

Copyright © 2005 ARM Limited. All rights reserved.

Release Information

Change history

Date	Issue	Change
31 March 2005	A	First release

Proprietary Notice

Words and logos marked with ® or ™ are registered trademarks or trademarks owned by ARM Limited, except as otherwise stated bellow in this proprietary notice. Other brands and names mentioned herein may be the trademarks of their respective owners.

Neither the whole nor any part of the information contained in, or the product described in, this document may be adapted or reproduced in any material form except with the prior written permission of the copyright holder.

The product described in this document is subject to continuous developments and improvements. All particulars of the product and its use contained in this document are given by ARM in good faith. However, all warranties implied or expressed, including but not limited to implied warranties of merchantability, or fitness for purpose, are excluded.

This document is intended only to assist the reader in the use of the product. ARM Limited shall not be liable for any loss or damage arising from the use of any information in this document, or any error or omission in such information, or any incorrect use of the product.

Confidentiality Status

This document is Confidential. This document may only be used and distributed in accordance with the terms of the agreement entered into by ARM and the party that ARM delivered this document to.

Product Status

The information in this document is final, that is for a developed product.

Web Address

`http://www.arm.com`

Contents
ARM7TDMI Integration Manual

Copyright © 2005 ARM Limited. All rights reserved.
Confidential

Copyright © 2005 ARM Limited. All rights reserved.
Confidential

List of Tables
ARM7TDMI Integration Manual

Copyright © 2005 ARM Limited. All rights reserved.
Confidential

Copyright © 2005 ARM Limited. All rights reserved.
Confidential

List of Figures
ARM7TDMI Integration Manual

Copyright © 2005 ARM Limited. All rights reserved.
Confidential

Copyright © 2005 ARM Limited. All rights reserved.
Confidential

Preface

This preface introduces the *ARM7TDMI Revision Rev3a and r3p0 Integration Manual* (IM). It contains the following sections:

Copyright © 2005 ARM Limited. All rights reserved.
Confidential

About this manual

The purpose of this manual is to provide all the information for integrating an ARM7TDMI macrocell into your *System on Chip* (SoC) design. It does not provide details on how to run specific tools to perform a specific task.

Note

- In this manual the term macrocell refers to the ARM7TDMI macrocell, unless explicit reference is made to other macrocells.
- The term processor refers to the ARM7TDMI processor unless explicit reference is made to other processors.
- The ARM7TDMI r3p0 is the same as theARM7TDMI Rev3a.

Product revision status

The r*n*p*n* identifier indicates the revision status of the product described in this manual, where:

r*n* Identifies the major revision of the product.

p*n* Identifies the minor revision or modification status of the product.

Intended audience

This manual is written for system designers, system integrators, and verification engineers who are integrating an ARM7TDMI macrocell in a *System-on-Chip* (SoC) device.

Using this manual

This document is organized into the following chapters:

Chapter 1 *Introduction*

Read this chapter for a general description of integration, a top-level description of the macrocell, supported integration design flow, and the deliverables supplied by ARM Limited.

Chapter 2 *Key Integration Points*

Read this chapter for a list of the key points that you must consider to integrate the macrocell into your SoC design.

Copyright © 2005 ARM Limited. All rights reserved.
Confidential

Chapter 3 *Functional Integration Guidelines*

Read this chapter for information on how to approach functional integration of the macrocell into your SoC design. This chapter provides information on clocks, resets, the macrocell interfaces, and miscellaneous signals.

Chapter 4 *DFT Guidelines*

Read this chapter for information on how to approach *Design For Test* (DFT) of the macrocell in your SoC design. This chapter provides information on scan chains, test shadows, the production test modes, and the test interface.

Chapter 5 *Functional Verification*

Read this chapter for information on how to simulate the macrocell in your SoC design, and how to use the simulation and sign-off models for the macrocell.

Chapter 6 *DFT Verification*

Read this chapter for information on how to convert and replay the macrocell test vectors in your SoC design.

Chapter 7 *Design Synthesis*

Read this chapter for information on how to synthesize your SoC RTL with the macrocell.

Chapter 8 *Physical Integration Guidelines*

Read this chapter for information on how to approach physical integration of the macrocell into your SoC design, and for information on the place and route process and checks on the physical layout.

Chapter 9 *Post-Layout Verification*

Read this chapter for information on how to verify the macrocell in your SoC design using:

- static verification by *Logical Equivalence Checking* (LEC) and *Static Timing Analysis* (STA)
- dynamic verification.

Chapter 10 *Sign Off*

Read this chapter for information on how to do the technical sign-off of the macrocell in your SoC design.

Copyright © 2005 ARM Limited. All rights reserved.
Confidential

Conventions

Conventions that this manual can use are described in:

- *Typographical*
- *Timing diagrams*
- *Signals* on page xiii
- *Numbering* on page xiii.

Typographical

The typographical conventions are:

italic Highlights important notes, introduces special terminology, denotes internal cross-references, and citations.

bold Highlights interface elements, such as menu names. Denotes ARM processor signal names. Also used for terms in descriptive lists, where appropriate.

`monospace` Denotes text that you can enter at the keyboard, such as commands, file and program names, and source code.

`mono`space Denotes a permitted abbreviation for a command or option. You can enter the underlined text instead of the full command or option name.

`monospace italic` Denotes arguments to monospace text where the argument is to be replaced by a specific value.

`monospace bold` Denotes language keywords when used outside example code.

< and > Angle brackets enclose replaceable terms for assembler syntax where they appear in code or code fragments. They appear in normal font in running text. For example:

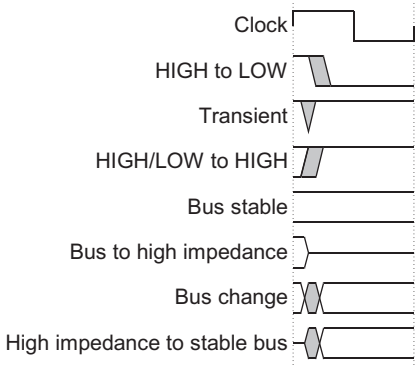
- `MRC p15, 0 <Rd>, <CRn>, <CRm>, <Opcode_2>`
- The Opcode_2 value selects the register to access.

Timing diagrams

This manual contains one or more timing diagrams. The figure named *Key to timing diagram conventions* on page xiii explains the components used in timing diagrams. When variations occur they have clear labels. You must not assume any timing information that is not explicit in the diagrams.

Copyright © 2005 ARM Limited. All rights reserved.
Confidential

Shaded bus and signal areas are undefined, so the bus or signal can assume any value within the shaded area at that time. The actual level is unimportant and does not affect normal operation.



Key to timing diagram conventions

Signals

The signal conventions are:

Signal level The level of an asserted signal depends on whether the signal is active-HIGH or active-LOW. Asserted means HIGH for active-HIGH signals and LOW for active-LOW signals:

Prefix H Denotes *Advanced Microcontroller Bus Architecture* (AMBA™) *Advanced High-performance Bus* (AHB) signals.

Prefix n Denotes active-LOW signals except in the case of AHB or *Advanced Peripheral Bus* APB reset signals. These are named **HRESETn** and **PRESETn** respectively.

Prefix P Denotes AMBA APB signals.

Numbering

The numbering convention is:

<size in bits>'<base><number>

This is a Verilog method of abbreviating constant numbers. For example:

- 'h7B4 is an unsized hexadecimal value.
- 'o7654 is an unsized octal value.

Copyright © 2005 ARM Limited. All rights reserved.
Confidential

- 8’d9 is an eight-bit wide decimal value of 9.
- 8’h3F is an eight-bit wide hexadecimal value of `0x3F`. This is equivalent to b00111111.
- 8’b1111 is an eight-bit wide binary value of b00001111.

Further reading

This section lists publications by ARM Limited, and by third parties.

ARM Limited periodically provides updates and corrections to its documentation. See `http://www.arm.com` for current errata sheets, addenda, and the ARM Limited Frequently Asked Questions list.

ARM publications

This manual contains information that is specific to the ARM7TDMI hardened macrocell. See the following documents for other relevant information:

- *ARM7TDMI Core Configuration and Performance Summary*

  Note

  The contents of this document depend on the chosen process technology.

- *ARM7TDMI Rev3 Technical Reference Manual* (ARM DDI 0029)
- *RealView ICE® User Guide* (ARM DUI 0155)
- *Application Note 31: Using EmbeddedICE* (ARM DAI 0031)
- *ETM7 Technical Reference Manual* (ARM DDI 0158)
- *AMBA™ Rev 2.0 Specification* (ARM IHI 0011)
- *Application Note 99: Core Type and Revision Identification* (ARM DAI 0099)
- *Design Simulation Model User Guide* (ARM DUI 0302)
- *RVCT 2.0 Assembler Guide* (ARM DUI 0204)
- *AHB CPU Wrappers Technical Reference Manual* (ARM DDI 0169).

Other publications

This section lists relevant documents published by third parties:

- Steve Furber, *ARM System-on-Chip Architecture*, (2nd edition 2000), Addison Wesley, ISBN 0-201-67519-6.

  Note

  This book is available in several languages.

Copyright © 2005 ARM Limited. All rights reserved.
Confidential

Feedback

ARM Limited welcomes feedback on the ARM7TDMI macrocell and its documentation.

Feedback on the product

If you have any comments or suggestions about this product, contact your supplier giving:

- the product name
- a concise explanation of your comments.

Feedback on this manual

If you have any comments on this manual, send email to `errata@arm.com` giving:

- the title
- the number
- the page number(s) to which your comments apply
- a concise explanation of your comments.

ARM Limited also welcomes general suggestions for additions and improvements.

Copyright © 2005 ARM Limited. All rights reserved.
Confidential

Copyright © 2005 ARM Limited. All rights reserved.
Confidential

Chapter 1
Introduction

This chapter describes the overall integration process and the macrocell. It contains the following sections:

Copyright © 2005 ARM Limited. All rights reserved.
Confidential

1.1 About integration

Integration is the final major phase in the process for inclusion of the macrocell in your SoC design. The other phases include the creation and implementation of the macrocell RTL. Integration involves:

- connection of the macrocell to all the necessary peripherals and buses
- implementation of the chosen test strategies for the macrocell
- verification of the macrocell within the SoC design.

For a general description of the integration flow, see *Typical integration design flow* on page 1-8.

The elements that you include have implications for your design. You must consider:

- interfaces, especially the treatment of unused signals
- special cases and configuration such as memory sizes
- *Design For Test* (DFT) strategies
- physical integration and layout
- verification.

You can take one of the following approaches to integrate the macrocell in to your SoC design:

- use an AHB wrapper, this is recommended
- use the native memory interface.

For guidance on the choice of approach to take, see *Approaches to integration* on page 1-3.

You might have different elements in your SoC design, depending on the approach you take, but the main connections to the macrocell are:

- AHB interfaces conforming to AMBA, for the AHB Wrapper approach only
- memory interfaces
- coprocessor interface
- JTAG interface.

You also have to integrate:

- clocks and resets
- power connections
- debug signals
- miscellaneous signals, such as interrupts.

Copyright © 2005 ARM Limited. All rights reserved.
Confidential

1.1.1 Approaches to integration

There are two approaches that you can take to integrate the macrocell into your SoC design:

- *AHB wrapper*
- *Native memory interface* on page 1-4.

AHB wrapper

In this approach you use an AHB wrapper around the macrocell to connect peripherals. This is the preferred method. For details of the AHB wrapper, see the *AHB CPU Wrappers Technical Reference Manual*.

Use this approach when you require the extra functionality that the AHB wrapper provides.

The advantages of this approach are:

- simpler connection to a wide range of peripherals
- easier reuse of IP blocks
- fewer manufacturing test patterns, approximately 150 000 cycles
- easier to synthesize because the AHB interface works off a single rising edge of the clock.

The main disadvantage of this approach is:

- Potentially higher latency than the native memory interface because the wrapper adds a wait state for every non-sequential (NSEQ) transfer in a burst.

  Depending on the timing of your design, the native memory interface might also show this disadvantage at high clock frequencies.

Figure 1-1 on page 1-4 shows some simple SoC configurations for the macrocell with an AHB wrapper.

Copyright © 2005 ARM Limited. All rights reserved.
Confidential

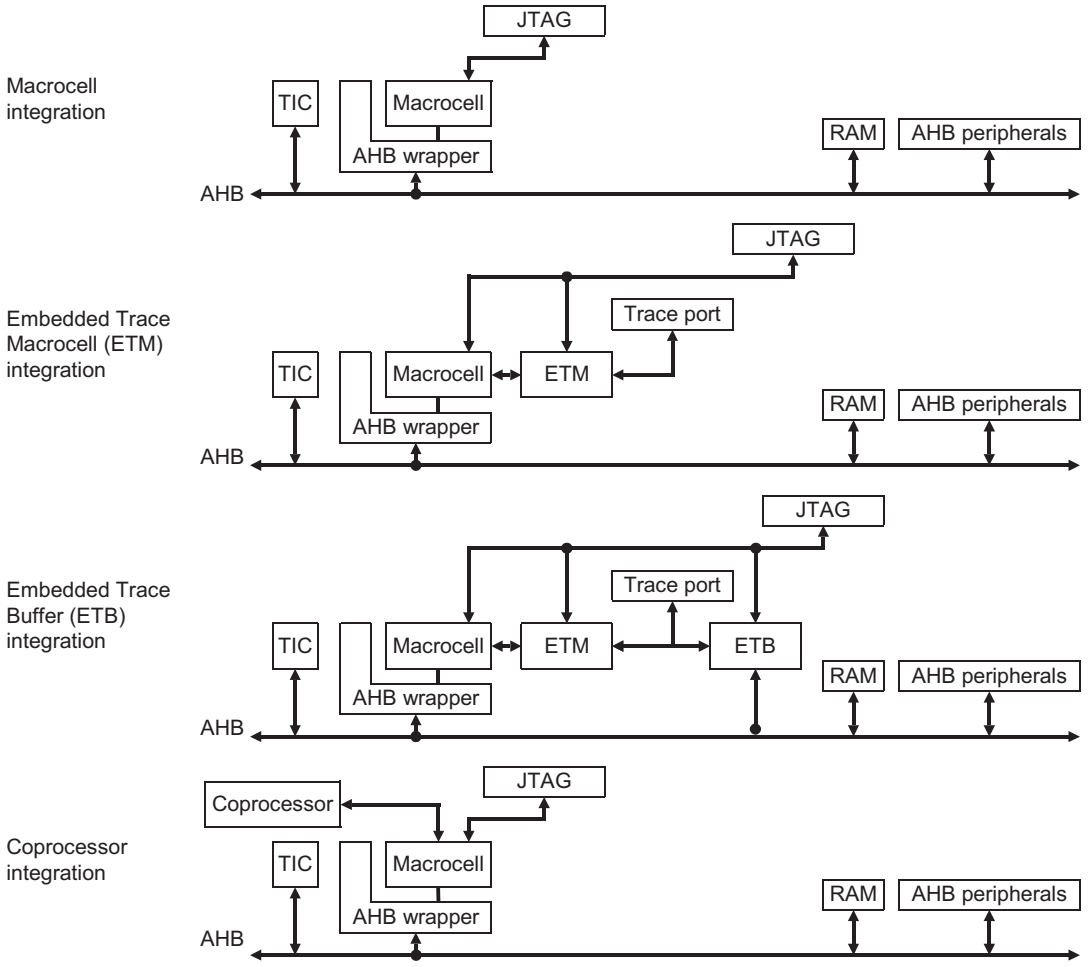


Figure 1-1 Simple SoC integration examples with an AHB wrapper

Native memory interface

In this approach you connect peripherals directly to the native memory interface and do not use an AHB wrapper.

Use this approach when you do not include any of the functionality that the AHB wrapper supports and you require the smallest possible SoC.

Copyright © 2005 ARM Limited. All rights reserved.
Confidential

The advantages of the native memory interface are:

- small design size
- potentially lower latency operation than with the AHB wrapper, depending on the operating frequency of your design
- support for bi-directional bus for the SoC infrastructure.

The disadvantages of the native memory interface are:

- added complexity of interface connections
- more complicated macrocell native memory interface
- more manufacturing test patterns, approximately 1 000 000 cycles
- more complicated to synthesize in the SoC because the native interface works on both rising and falling clock edges.

Figure 1-2 on page 1-6 shows some simple SoC configurations for the macrocell with the native memory interface.

Copyright © 2005 ARM Limited. All rights reserved.
Confidential

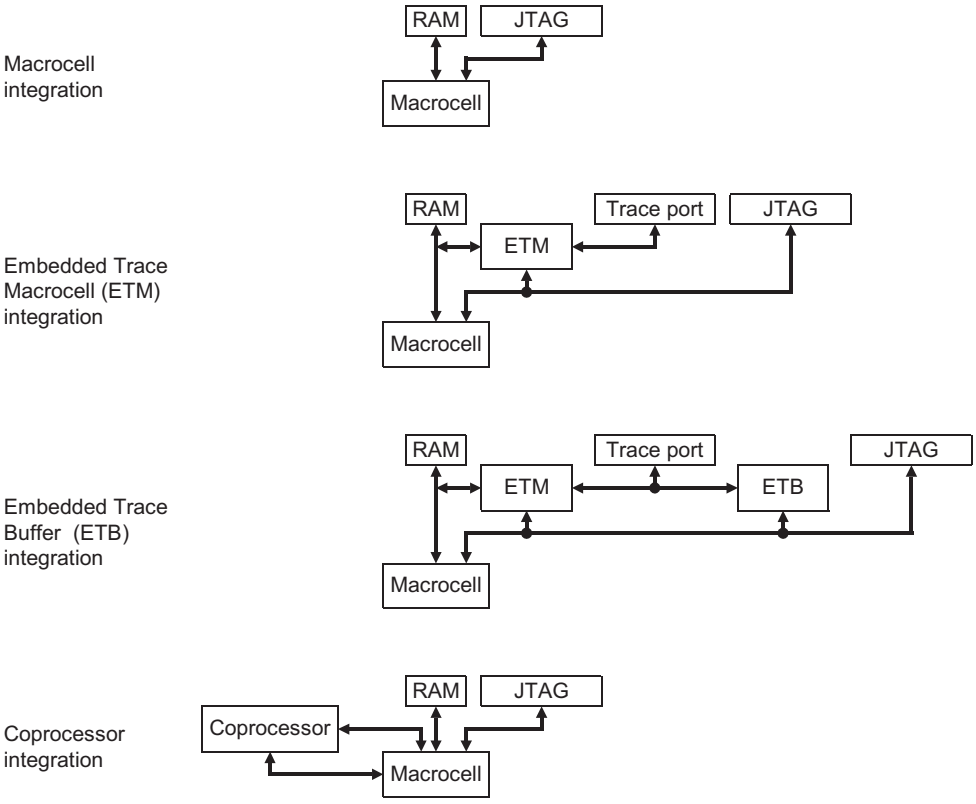


Figure 1-2 Simple SoC integration examples with the native memory interface

Copyright © 2005 ARM Limited. All rights reserved.
Confidential

1.2 About the macrocell

The ARM7TDMI macrocell is an implementation of the ARM7 processor core, see the *ARM7TDMI Rev3 Technical Reference Manual* for details.

For specific details relating to your macrocell configuration, see the *ARM7TDMI Core Configuration and Performance Summary* document.

The macrocell has passed the following tests in isolation:

- *Layout Versus Schematic* (LVS) checks
- *Design Rules Check* (DRC)
- antenna rules check
- timing checks.

Copyright © 2005 ARM Limited. All rights reserved.
Confidential

1.3 Typical integration design flow

This document assumes that you follow a typical SoC design flow. Figure 1-3 on page 1-9 shows a simplified version of this design flow.

Note

The *Automatic Test Pattern Generation* (ATPG) stage appears as an output from the verification of the synthesized netlist process but it can also be an output of the verification of the post-layout netlist process.

Copyright © 2005 ARM Limited. All rights reserved.
Confidential

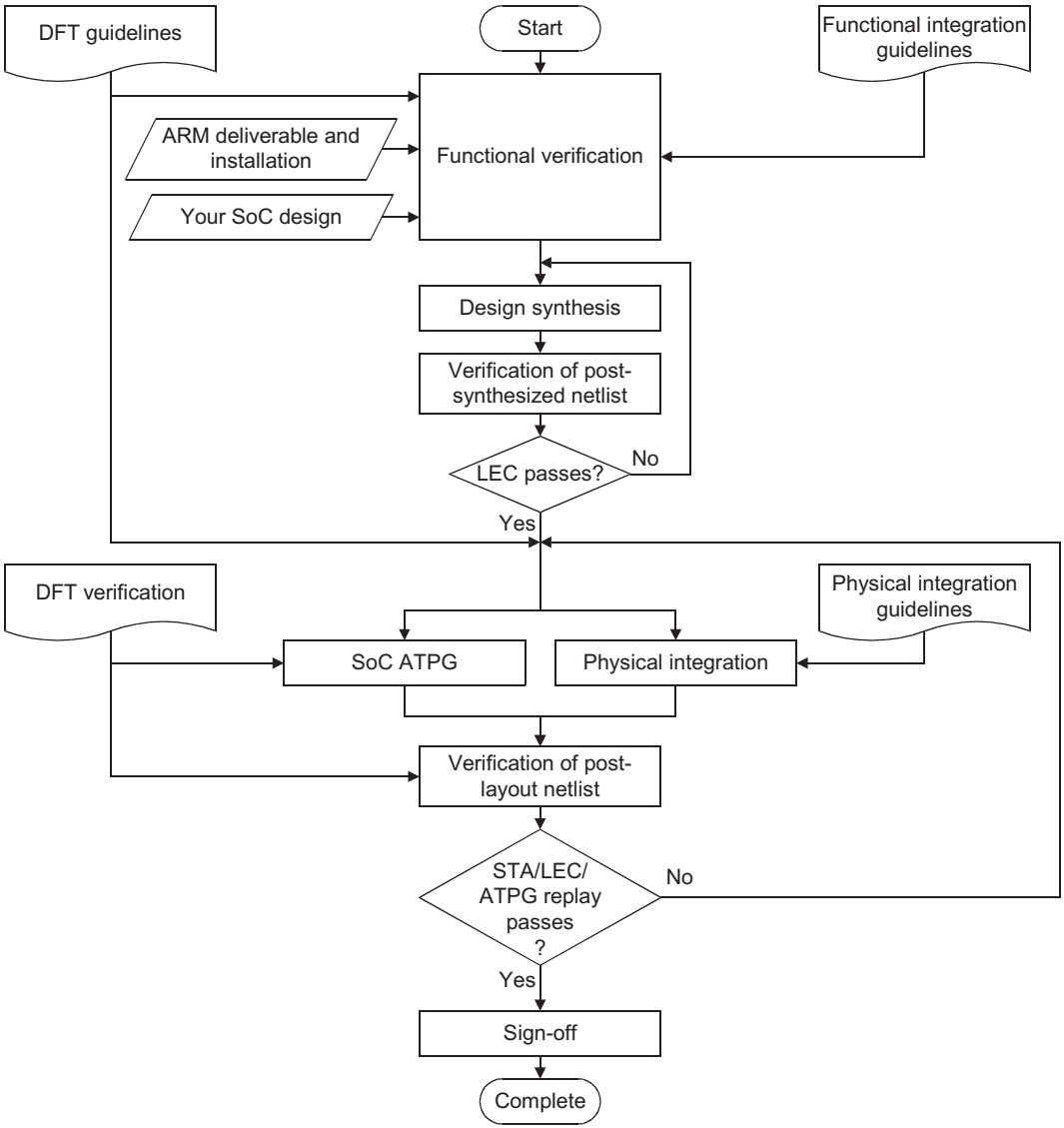


Figure 1-3 Integration design flow

Copyright © 2005 ARM Limited. All rights reserved.
Confidential

Table 1-1 shows the chapters in this manual that relate to the document inputs shown in Figure 1-3 on page 1-9.

Table 1-1 Integration flow cross references

Input	Reference
Functional integration guidelines	see Chapter 3 *Functional Integration Guidelines*
DFT guidelines	see Chapter 4 *DFT Guidelines*
DFT verification	see Chapter 6 *DFT Verification*
Physical integration guidelines	see Chapter 8 *Physical Integration Guidelines*

Copyright © 2005 ARM Limited. All rights reserved.
Confidential

1.4 ARM deliverables

Unpack the deliverables in accordance with your release note. Before starting you must make sure that the unpacked deliverables have the correct data structure.

Copyright © 2005 ARM Limited. All rights reserved.
Confidential

1.5 Timing parameters

The *ARM7TDMI Rev3 Technical Reference Manual* defines the timing parameters for all input and output signals.

Copyright © 2005 ARM Limited. All rights reserved.
Confidential

Chapter 2
Key Integration Points

This chapter describes the key integration points for the macrocell. It contains the following sections:

Copyright © 2005 ARM Limited. All rights reserved.
Confidential

2.1 About key integration points

This chapter contains a list of the main points to consider while integrating the macrocell in your SoC design with either an AHB wrapper or the native memory interface. You must read this chapter in conjunction with the detailed information in the rest of this manual and the information referred to in *Further reading* on page xiv.

You can use *Key points for the AHB wrapper* on page 2-3 or *Key points for the native memory interface* on page 2-4 as a general check that you have covered the integration steps described in the other chapters, but you must complete all the integration steps as they are described in the later chapters.

Note

You must read the relevant sections in this manual for details of the key points.

Copyright © 2005 ARM Limited. All rights reserved.
Confidential

2.2 Key points for the AHB wrapper

Table 2-1 shows key integration tasks for macrocell integration.

Table 2-1 Key integration tasks for the AHB wrapper

Key task		Things you must know about this task
1.	Generate clocks and resets correctly	See *Clocks and resets* on page 3-6
2.	Separate **HRESETn** and **nTRST** to enable debugging at reset	See *Reset* on page 3-7
3.	Deassert **HRESETn**, and **nTRST** correctly	See *Clocks and resets* on page 3-6
4.	During reset, hold **HRESETn** LOW for at least three **HCLK** cycles	See *Clocks and resets* on page 3-6
5.	During power-on reset, hold **DBGnTRST** LOW for at least one **HCLK** cycle	See *Clocks and resets* on page 3-6
6.	Ensure that you tie coprocessor handshake signals **CPA** and **CPB** HIGH if there is no coprocessor present	See *Coprocessor interface* on page 3-17
7.	Ensure that you tie **DBGEN** HIGH to enable full debug facilities	See *Debug interface* on page 3-26
8.	Tie off or connect all macrocell inputs appropriately	See Chapter 3 *Functional Integration Guidelines*
9.	Install the *Design Simulation Model* (DSM) to verify the SoC design	See *Using the DSM* on page 5-3
10.	Replay and capture the test vectors at the SoC level	See *Capturing TIC vectors* on page 6-6
11.	Compile the macrocell timing view	See *Using the timing models* on page 7-3
12.	Set up your synthesis and *Static Timing Analysis* (STA) environments to use the compiled timing views of the macrocell	See *Using the timing models* on page 7-3 and *Static Timing Analysis* on page 9-5
13.	Use STA and LEC for post-layout verification	See Chapter 9 *Post-Layout Verification*
14.	Connect all power and ground pins correctly during floorplanning	See *Power connections* on page 8-6
15.	Place the macrocell observing all requirements and considering all implications	See *Placement* on page 8-3
16.	Conform to the macrocell routing restrictions	See *Routing restrictions* on page 8-9

Copyright © 2005 ARM Limited. All rights reserved.
Confidential

2.3 Key points for the native memory interface

Table 2-2 shows key integration tasks for macrocell integration.

Table 2-2 Key integration tasks for the native memory interface

Key task		Things you must know about this task
1.	Generate clocks and resets correctly	See *Clocks and resets* on page 3-6
2.	Separate **nRESET** and **nTRST** to enable debugging at reset	See *Reset* on page 3-7
3.	Deassert **nRESET**, and **nTRST** correctly	See *Clocks and resets* on page 3-6
4.	During reset, hold **nRESET** LOW for at least two **MCLK** cycles	See *Clocks and resets* on page 3-6
5.	During power-on reset, hold **DBGnTRST** LOW for at least one **MCLK** cycle	See *Clocks and resets* on page 3-6
6.	Tie coprocessor handshake signals **CPA** and **CPB** HIGH if there is no coprocessor present	See*Coprocessor interface* on page 3-17
7.	Tie **DBGEN** HIGH to enable full debug facilities	See *Debug interface* on page 3-26
8.	Tie off or connect all macrocell inputs appropriately	See Chapter 3 *Functional Integration Guidelines*
9.	Install the *Design Simulation Model* (DSM) to verify the SoC design	See *Using the DSM* on page 5-3
10.	Replay, and possibly capture, the test vectors at the SoC level	See *Capturing TIC vectors* on page 6-6
11.	Compile the macrocell timing view	See *Using the timing models* on page 7-3
12.	Set up your synthesis and *Static Timing Analysis* (STA) environments to use the compiled timing views of the macrocell	See *Using the timing models* on page 7-3 and *Static Timing Analysis* on page 9-5
13.	Use STA and LEC for post-layout verification	See Chapter 9 *Post-Layout Verification*
14.	Connect all power and ground pins correctly during floorplanning	See *Power connections* on page 8-6
15.	Place the macrocell observing all requirements and considering all implications	See *Placement* on page 8-3
16.	Conform to the macrocell routing restrictions	See *Routing restrictions* on page 8-9

Copyright © 2005 ARM Limited. All rights reserved.
Confidential

Chapter 3
Functional Integration Guidelines

This chapter describes the functional integration requirements of the macrocell in your SoC design. It contains the following sections:

Copyright © 2005 ARM Limited. All rights reserved.
Confidential

3.1 About functional integration

This chapter contains information that you must consider before or during the integration of the macrocell into your SoC design. The chapter only provides information about functional integration of the macrocell. See Chapter 8 *Physical Integration Guidelines* for information on physical integration.

The integration of the macrocell into your SoC design depends on whether or not you use the AHB wrapper, see *Approaches to integration* on page 1-3. The preferred approach is to use the AHB wrapper, see the *AHB CPU Wrappers Technical Reference Manual*. Figure 3-1 on page 3-3 shows the interfaces for the macrocell with the AHB wrapper and Figure 3-2 on page 3-4 shows the native memory interface without the AHB wrapper.

Copyright © 2005 ARM Limited. All rights reserved.
Confidential

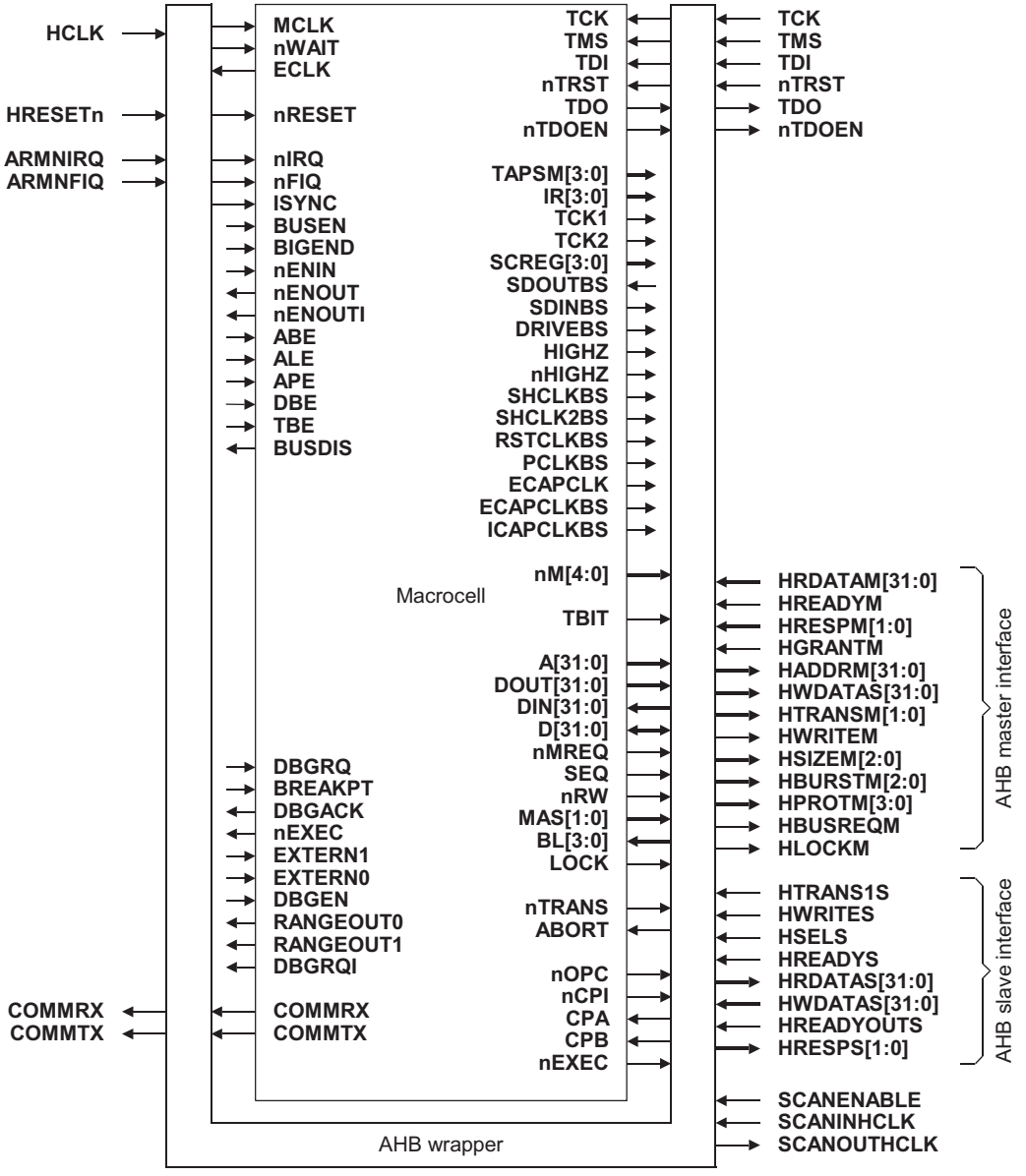


Figure 3-1 Macrocell signals with the AHB wrapper

Copyright © 2005 ARM Limited. All rights reserved.
Confidential

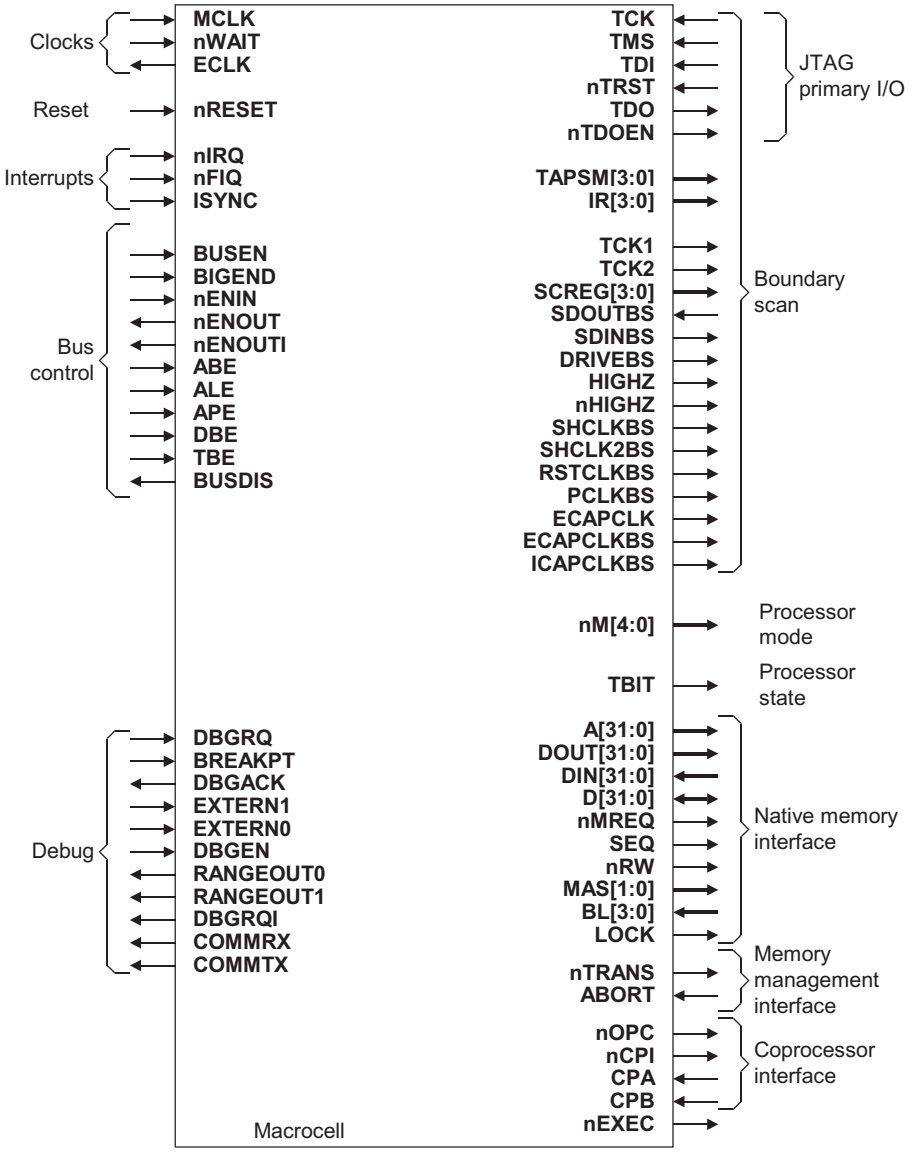


Figure 3-2 Macrocell signals with the native memory interface

Table 3-1 on page 3-5 shows the corresponding interfaces for integration for each approach.

Copyright © 2005 ARM Limited. All rights reserved.
Confidential

Table 3-1 Integration with the AHB wrapper interface or with the native memory interface

Interface	With AHB wrapper	With native memory interface
Clocks and resets	See *Clocks and resets for the AHB wrapper* on page 3-6	See *Clocks and resets for the native memory interface* on page 3-6
AHB interface	See *AHB wrapper interface* on page 3-14	-
Native memory interface	-	See *Native memory interface* on page 3-10
Coprocessor interface	See *AHB wrapper with coprocessors* on page 3-17	See *Native memory interface with coprocessors* on page 3-18
Miscellaneous signals	Common to AHB wrapper and native memory interface, see *Miscellaneous signals* on page 3-20	
JTAG interface	Common to AHB wrapper and native memory interface, see *JTAG interface* on page 3-23	
Debug interface	Common to AHB wrapper and native memory interface, see *Debug interface* on page 3-26	
Embedded Trace Macrocell (ETM) interface	Common to AHB wrapper and native memory interface, see *Embedded Trace Macrocell interface* on page 3-28	

Copyright © 2005 ARM Limited. All rights reserved.
Confidential

3.2 Clocks and resets

This section describes how to connect the macrocell clocks and resets in your SoC design, see also *Debug clocks* on page 3-27.

3.2.1 Clocks and resets for the AHB wrapper

Table 3-2 shows the clock and reset signals present on the AHB wrapper, and how to use these signals in your SoC design. For details, see the *AHB CPU Wrappers Technical Reference Manual*.

Table 3-2 Clock and reset signals for the AHB wrapper

Signal name	Direction	Description	Connection notes
HCLK	Input	Bus clock	This signal times all bus transfers. All signal timings are relative to the rising edge of **HCLK**.
HRESETn	Input	Active-LOW system reset signal	To ease integration into an AMBA AHB system you must ensure that this signal is held LOW for a minimum of three **HCLK** cycles, and deasserted synchronously. See *Reset* on page 3-7.

3.2.2 Clocks and resets for the native memory interface

Table 3-2 shows the system clock and reset signals present in the macrocell, and how to use these signals in your SoC design.

Table 3-3 Clock and reset signals for the native memory interface

Signal name	Direction	Description	Connection notes
MCLK	Input	Memory clock	Main clock for all memory and processor operations.
nRESET	Input	Reset signal	The processor restarts from address 0 when **nRESET** is HIGH for at least one cycle. You must ensure that this signal is held LOW for a minimum of two **MCLK** cycles with **nWAIT** HIGH. See *Reset* on page 3-7.
nWAIT	Input	Wait signal	Extends bus access over more than one **MCLK** cycle. **nWAIT** must only change when **MCLK** is LOW. If your design does not use this signal, then you must tie it HIGH.
ECLK	Output	External clock output	Equivalent to the internal clock for the main processor core.

Copyright © 2005 ARM Limited. All rights reserved.
Confidential

When you design the clock for the macrocell with the native memory interface, you must not violate the minimum HIGH pulse width or the minimum LOW pulse width. For example, Figure 3-3 shows the clock for a macrocell characterized with a maximum frequency of 90MHz. For a 50 percent duty cycle, the minimum LOW and HIGH pulse width is 5.55ns. For a 32MHz clock, 31.25ns period, you can vary the duty cycle anywhere between the two extremes in Figure 3-3.

Note

The macrocell uses both the rising and the falling edge of the clock, therefore you must consider both the HIGH and LOW pulse widths.

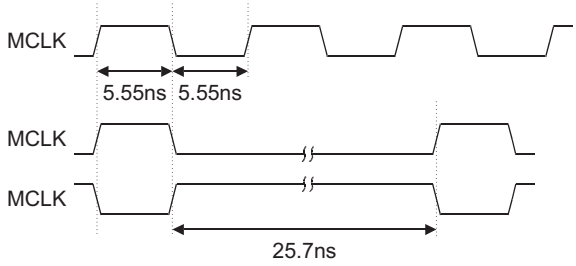


Figure 3-3 Macrocell clock example for the native memory interface

For more detail on modulating **MCLK** and the use of wait states to control bus cycles see *Stretching access times* in the *Memory Interface* chapter of the *ARM7TDMI Rev3 Technical Reference Manual*.

3.2.3 Reset

It is necessary to reset the processor immediately on power-up. This removes any undefined conditions in the processor that might combine to cause a DC path and increase current consumption.

Note

- Your system must assert **nRESET** for a minimum of two **MCLK** cycles to fully reset the core. The system must also set **nTRST** LOW for at least T_{bsr}, to reset the Embedded ICE Logic and the TAP controller, even if you do not use the debug features.
- Your system must hold **nWAIT** HIGH during the reset sequence.

Copyright © 2005 ARM Limited. All rights reserved.
Confidential

You can use the **HRESETn** or **nRESET**, and **nTRST** signals in the macrocell to reset the core and debug parts of the processor core independently. It is recommended that you separate the connections for the core reset and debug reset signals to enable debugging at the reset condition.

Figure 3-4 shows how the system can assert **HRESETn** or **nRESET**, and **nTRST** asynchronously to the clock signal **CLK** but must deassert them synchronously to **CLK**. **HRESETn** and **nRESET** synchronize in the same way regardless of the design approach you take.

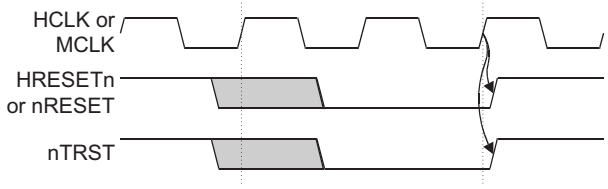


Figure 3-4 Reset signal synchronization

The reset synchronization must enable the tester to control the reset signals, **HRESETn** or **nRESET**, and **nTRST**, during production test. Figure 3-5 shows an example circuit for synchronization to meet the production test requirements.

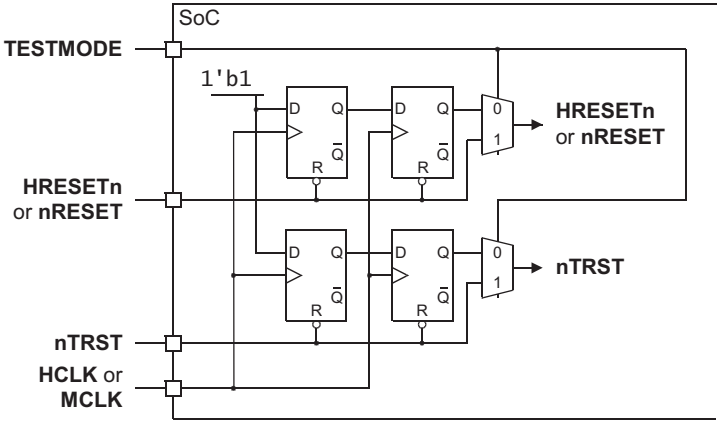


Figure 3-5 Reset synchronization circuit

Copyright © 2005 ARM Limited. All rights reserved.
Confidential

Table 3-4 shows the reset signaling combinations and possible applications.

Table 3-4 Reset modes

Reset mode	HRESETn or nRESET	nTRST	Application
Power-on reset	0	0	Reset at power up, full system reset.
Processor core reset	0	1	Reset of processor core only, watchdog reset.
Debug logic reset	1	0	Reset of the debug logic in the macrocell.
Normal	1	1	No reset. Normal run mode.

For more information on device reset, see the *ARM7TDMI Rev3 Technical Reference Manual*.

Note

- **HRESETn** or **nRESET** must be asserted LOW for a minimum of three **HCLK** or two **MCLK** cycles.
- **nTRST** is asserted LOW for a minimum of one **HCLK** or **MCLK** cycle.
- **nTRST** must be asserted LOW at power-on reset.

Copyright © 2005 ARM Limited. All rights reserved.
Confidential

3.3 Native memory interface

For more information on the different transfers made by the macrocell see the *ARM7TDMI Rev3 Technical Reference Manual*.

Table 3-5 to Table 3-8 on page 3-12 show the native memory interface signals present on the macrocell, and describe how you must set the signals in your SoC design when you do not include the AHB wrapper. See the *ARM7TDMI Rev3 Technical Reference Manual* for details of these signals.

Table 3-5 shows the address cycle signals.

Table 3-5 Address cycle signals

Signal Name	Direction	Description
nMREQ	Output	Indicates when the core performs: an external memory access, **nMREQ** = 0 an internal or co-processor cycle, **nMREQ** = 1.
SEQ	Output	Indicates when the address on the next cycle is: a sequential address cycle, **SEQ** = 1 a non-sequential address cycle, **SEQ** = 0.

Note

The core drives **nMREQ** and **SEQ** in the previous cycle of a memory access to indicate the type of transfer in the next cycle.

Copyright © 2005 ARM Limited. All rights reserved.
Confidential

Table 3-6 shows the addressing and control signals.

Table 3-6 Addressing and control signals

Signal Name	Direction	Description
A[31:0]	Output	Contains the address output from core
nRW	Output	Indicates when the core performs: a read, **nRW** = 0 a write, **nRW** = 1
MAS[1:0]	Output	Indicates when the core performs: a byte access, **MAS** = 2'b00 a halfword access, **MAS** = 2'b01 a word access, **MAS** = 2'b10 **MAS** = 2'b11 is reserved
LOCK	Output	**LOCK** = 1 during the time when the core performs a locked transfer, because of the execution of a SWP instruction in the core
nTRANS	Output	Indicates when the core performs an access in: a privileged mode, **nTRANS** = 1 a non-privileged mode, **nTRANS** = 0
nOPC	Output	Indicates when the core performs an access as a result of: an instruction fetch, **nOPC** = 0 a data transfer, **nOPC** = 1
TBIT	Output	Indicates when the core is in: ARM state, TBIT = 0 Thumb state, TBIT = 1

Typically an external memory system uses the combination of **nTRANS** and **nOPC** to determine a memory access and to check memory protection, for example to flag an abort based on an instruction fetch to a region of data memory.

Copyright © 2005 ARM Limited. All rights reserved.
Confidential

Table 3-7 shows the data transfer signals.

Table 3-7 Data transfer signals

Signal Name	Direction	Description
D[31:0]	Bi-directional	Carries bi-directional memory data. This bus is selected when **BUSEN** = 0. When your design does not use this bus you must leave it unconnected and not tied to a static value. When your design uses this bidirectional bus consider the requirement for bus keepers.
DIN[31:0]	Input	Carries uni-directional memory data. These buses are selected when **BUSEN** = 1.
DOUT[31:0]	Output	
BL[31:0]	Input	Constructs memory accesses to external narrow memory to form a wider access within the core.
ABORT	Input	Flags aborts to the core signalled by the external memory system. When fetching an instruction, setting **ABORT** = 1 results in the core taking a prefetch abort exception if the instruction attempts execution. When transferring data, setting **ABORT** = 1 makes the core take a data abort exception.
nENIN	Input	Facilitates bus-turnaround in a tri-state based SoC design. For details see the *ARM7TDMI Rev3 Technical Reference Manual*.
nENOUT	Output	
nENOUTI	Output	

Table 3-8 shows the static configuration signals.

Table 3-8 Static configuration signals

Signal Name	Direction	Description
BUSEN	Input	Selects between: bi-directional data bus, **BUSEN** = 0, using **D[31:0]** uni-directional data bus, **BUSEN** = 1, using **DIN[31:0]** and **DOUT[31:0]**.
BIGEND	Input	Selects between memory modes: Little Endian, **BIGEND** = 0 Big Endian, **BIGEND** =1.
ALE	Input	Superseded by **APE**. You must set this pin to logic 1.

Copyright © 2005 ARM Limited. All rights reserved.
Confidential

Table 3-8 Static configuration signals (continued)

Signal Name	Direction	Description
APE	Input	Controls timing of address and control signals. This pin is provided for compatibility with memory systems that impose different address and data timing requirements. Pipelined operation, **APE** = 1, ensures that the address that the core generates is as early as possible in the memory access cycle. This is useful in systems that require significant address decoding such as SDRAM. This is the recommended configuration. Non-Pipelined operation, **APE** = 0, ensures that the core generates the address in the same cycle as the data transfer. This is a common requirement for memory such as ROM or SRAM.
ABE	Input	Selects address and control signals between: driven, **ABE** = 1 high-impedance, **ABE** = 0. During normal operation **ABE** must be 1.
DBE	Input	Selects data bus, **DOUT[31:0]**,**D[31:0]**, between: driven, **DBE** = 1 **D[31:0]** high-impedance and **DOUT[31:0]** static, **DBE** = 0. During normal operation **DBE** must be 1.
TBE	Input	Test Bus Enable signal. Provides the functionality of **ABE** and **DBE** with a single configuration pin. The normal configuration, **TBE** = 1, drives **DBE** = 1 and **ABE** = 1 to ensure that all buses are driven. Test Configuration, **TBE** = 0, drives **DBE** = 0 and **ABE** = 0 and drives all buses to high impedance. For normal operation **TBE** must be 1.

Copyright © 2005 ARM Limited. All rights reserved.
Confidential

3.4 AHB wrapper interface

This section describes the integration of the AHB wrapper signals with your SoC design. For information about integration of the wrapper with the macrocell see the *AHB CPU Wrappers Technical Reference Manual*.

3.4.1 About the AHB wrapper interface

The AMBA AHB wrapper interface permits integration of the processor into a SoC design that uses the AMBA AHB bus standard. The wrapper is provided for you as synthesizable Verilog RTL.

The main function of the wrapper is to make the processor core appear as an AHB bus master in your design. The wrapper features an AHB slave interface so that you can perform manufacturing test on the core using parallel test patterns that ARM Limited provide. The synthesis process for the wrapper logic inserts scan cells and you can you can use scan insertion and ATPG techniques to test the wrapper logic.

3.4.2 AHB wrapper interface components

Figure 3-6 on page 3-15 shows the key components of the AHB wrapper infrastructure. The component structure includes:

- The A7TDMI level of hierarchy that instances the ARM7TDMI processor and all of the wrapper logic that ARM Limited supply. This is the level of hierarchy that you instance in your SoC design.
- The AHB block that you must supply and develop to provide the normal AHB SoC infrastructure. You can use components from the *AMBA Design Kit* (ADK) as a starting point for this logic.
- The *Test Interface Controller* (TIC) block that ARM Limited supplies to convert the off-chip test bus to AHB transactions. You can use this block to apply, design independent, functional test code to the processor through the AHB slave interface of the wrapper. You can also use the TIC to test other AHB slave components, for example on-chip memory with functional *Built-In Self Test* (BIST).

Copyright © 2005 ARM Limited. All rights reserved.
Confidential

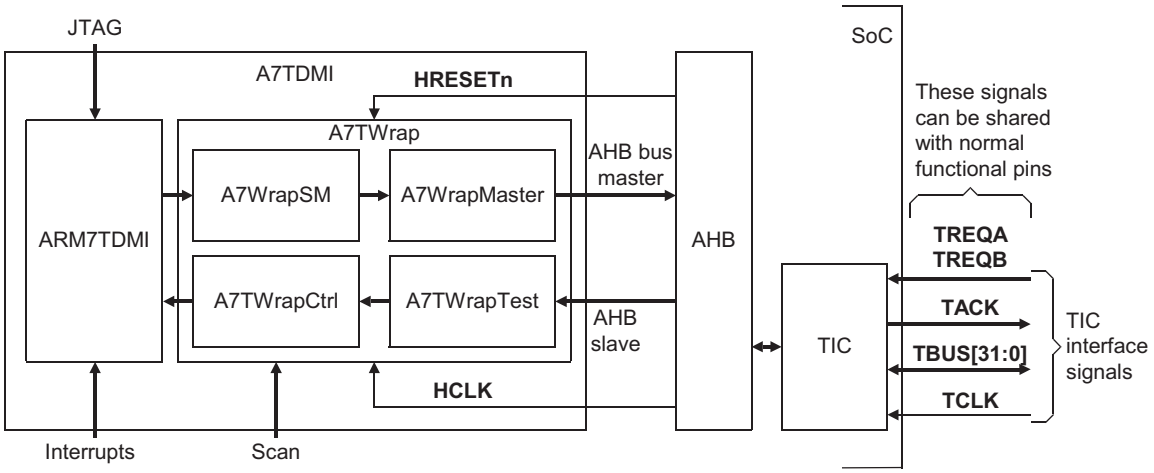


Figure 3-6 AMBA infrastructure

It is recommended that you understand the AHB bus master interface signals, see the *AMBA Specification,* before you connect any of the signals that this section describes. Table 3-9 shows the AMBA AHB bus master signals, and describes how to connect the signals in you SoC design.

Table 3-9 Bus master signals in the AMBA AHB wrapper

Signal name	Direction	Description	Connection information
HGRANTM	Input	Bus grant	Connects to the arbiter
HRDATAM[31:0]	Input	Read data bus	Connect from the slaves through AHB infrastructure
HREADYM	Input	Transfer complete	
HRESPM[1:0]	Input	Transfer response	
HTRANSM[1:0]	Output	Transfer type	Connect to the AHB arbiter and slaves through the bus infrastructure
HWRITEM	Output	Transfer direction	Connect to the slaves through the bus infrastructure
HSIZEM[2:0]	Output	Transfer size	
HBURSTM[2:0]	Output	Burst type	Connect to the AHB arbiter and slaves through the bus infrastructure
HPROTM[3:0]	Output	Protection control	Connect to the slaves through the bus infrastructure
HWDATAM[31:0]	Output	Write data bus	

Copyright © 2005 ARM Limited. All rights reserved.
Confidential

Table 3-9 Bus master signals in the AMBA AHB wrapper (continued)

Signal name	Direction	Description	Connection information
HLOCKM	Output	Request locked transfers	Connects to the arbiter
HADDRM[31:0]	Output	Address bus	Connect to address decoders, arbiter, and slaves through the bus infrastructure
HBUSREQM	Output	Bus request	Connects to the arbiter

For more details about the bus infrastructure required, see the *AMBA Specification*.

The wrapper features an AHB slave interface so that you can apply manufacturing test patterns to the core efficiently. Table 3-10 shows the AHB slave signals.

Table 3-10 Bus slave signals in the AMBA AHB wrapper

Signal name	Direction	Description	Connection information
HATRANS1S	Output	Transfer type	Connect to the AHB slave interface of the TIC block
HWRITES	Input	Transfer direction	
HWDATAS[31:0]	Input	Data bus in	
HSELS	Input	Slave interface select	
HREADYS	Input	Last transfer complete	
HRDATAS[31:0]	Output	Data bus out	
HREADYOUTS	Output	Transfer complete	
HRESPS[1:0]	Output	Transfer response	

3.4.3 Performance of the core with the wrapper

The AHB wrapper uses wait states for address synchronization. The wait states are only for Non-Sequential accesses and add approximately 33% more cycles for access to zero wait state memory.

The AHB wrapper adds a wait state for every NSEQ transfer in a burst.

Copyright © 2005 ARM Limited. All rights reserved.
Confidential

3.5 Coprocessor interface

This section describes the preferred approach for connection of coprocessors to the macrocell.

If you do not include a coprocessor in your SoC design, then you must tie off the processor core signals correctly. Table 3-11 shows the correct way to tie of unused signals when your system does not include any coprocessors.

For more information on signal timings or connection detail, see the *Coprocessor Interface* chapter in the *ARM7TDMI Rev3 Technical Reference Manual*.

Table 3-11 Coprocessor interface signals

Signal name	Direction	Description	Connection if no coprocessor connected
CPA	Input	Coprocessor handshake signals	Tie both these signals HIGH
CPB	Input		
nCPI	Output	Coprocessor instruction fetch signal	Leave these signals unconnected
nEXEC	Output	This signal indicates when the core executes the current instruction	

Note

- You must make sure that when **nRESET** is asserted **CPA** and **CPB** are HIGH.
- ETM7 connects to some coprocessor signals. If you use ETM, then you must provide for the signals appropriately, see *Embedded Trace Macrocell interface* on page 3-28.

3.5.1 AHB wrapper with coprocessors

Although the coprocessor control signals pass through the AHB wrapper, you must connect any coprocessors between the macrocell and the wrapper, for more detail see *Native memory interface with coprocessors* on page 3-18. Figure 3-7 on page 3-18 shows how coprocessor signals pass through the wrapper.

Copyright © 2005 ARM Limited. All rights reserved.
Confidential

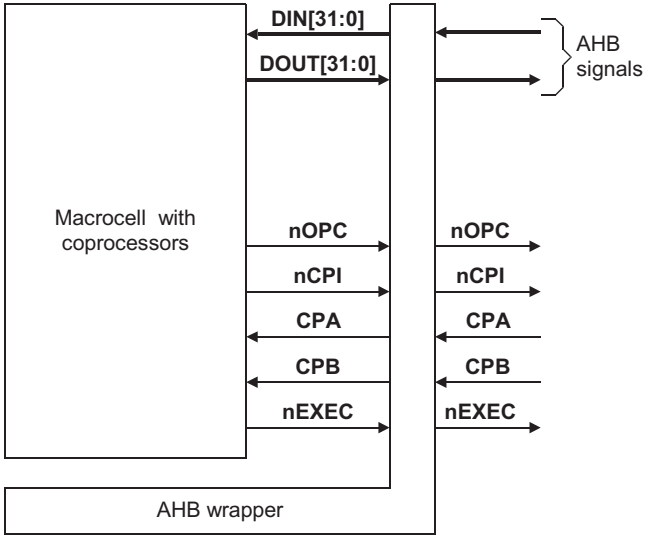


Figure 3-7 Coprocessor signals and the AHB wrapper

3.5.2 Native memory interface with coprocessors

The macrocell connects to coprocessors with the native memory interface together with these dedicated signals:

- **nCPI** coprocessor instruction
- **CPA** coprocessor absent
- **CPB** coprocessor busy.

For details of the macrocell coprocessor signals, see the *ARM7TDMI Rev3 Technical Reference Manual*.

Figure 3-8 on page 3-19 shows the connection of coprocessors to the macrocell. The diagram shows two methods for connection of data signals, one that uses the unidirectional buses and one that uses the bidirectional bus. You must select only one of these methods for all the coprocessors in your design, that corresponds to the data bus that you implement. For information on bus selection, see *Native memory interface* on page 3-10.

Note

When you use the AHB wrapper you must only use the uni-directional buses **DIN[31:0]** and **DOUT[31:0]**.

Copyright © 2005 ARM Limited. All rights reserved.
Confidential

The logic for the **ASEL** and **CSEL** signals is:

```
on FALLING MCLK
    ASEL = ((nMREQ = 1 and SEQ = 1) and (not nRW))
    CSEL = ((nMREQ = 1 and SEQ = 1) and (nRW))
```

If you use multiple coprocessors in your design, then you must multiplex the data signals.

When you do not have the AHB wrapper and use only one coprocessor you can connect **CPA**, **CPB**, and **nCPI** directly to the single coprocessor.

When you connect more than one coprocessor and do not have the AHB wrapper, or connect one or more coprocessors with the AHB wrapper you must:

- AND the **CPA** and **CPB** signals
- connect **nCPI** to each coprocessor.

Figure 3-8 shows the arrangement for connection of the **CPA**, **CPB**, and **nCPI** signals

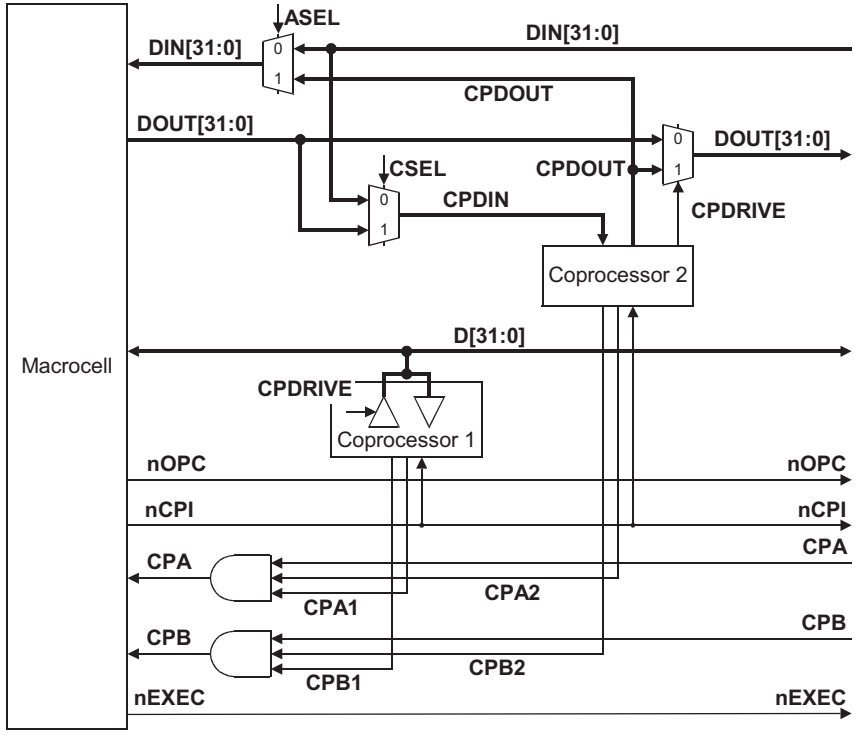


Figure 3-8 Connection of coprocessors

Copyright © 2005 ARM Limited. All rights reserved.
Confidential

3.6 Miscellaneous signals

Table 3-12 shows the miscellaneous signals present on the macrocell and how you must set the signals in your SoC design.

Table 3-12 Miscellaneous signals

Signal name		Direction	Description	Connection information
Native memory interface	**AHB interface**			
nFIQ	**ARMFIQ**	Input	Processor core interrupt signals, active-LOW.	Driven by interrupt sources or interrupt controller[a].
nIRQ	**ARMNIQ**	Input		
ISYNC	**Note** – Not present in AHB interface.	Input	When this signal is HIGH the processor synchronizes interrupts to the clock. When this signal is LOW the processor acts on asynchronous interrupts.	**Note** – For AHB systems that use the AHB wrapper **ISYNC** = 1.

a. See the ARM7TDMI Rev3 Technical Reference Manual.

Interrupts can be synchronous or asynchronous, depending on **ISYNC**. When interrupts are asynchronous, the processor synchronizes them internally before the core recognizes the interrupt.

Timing arcs exist for setup and hold times for **nIRQ** and **nFIQ** for both synchronous and asynchronous operation. In synchronous operation, your design must meet the setup and hold times with respect to the clock.

Figure 3-9 on page 3-21 shows the interrupt behavior for synchronous interrupts.

Copyright © 2005 ARM Limited. All rights reserved.
Confidential

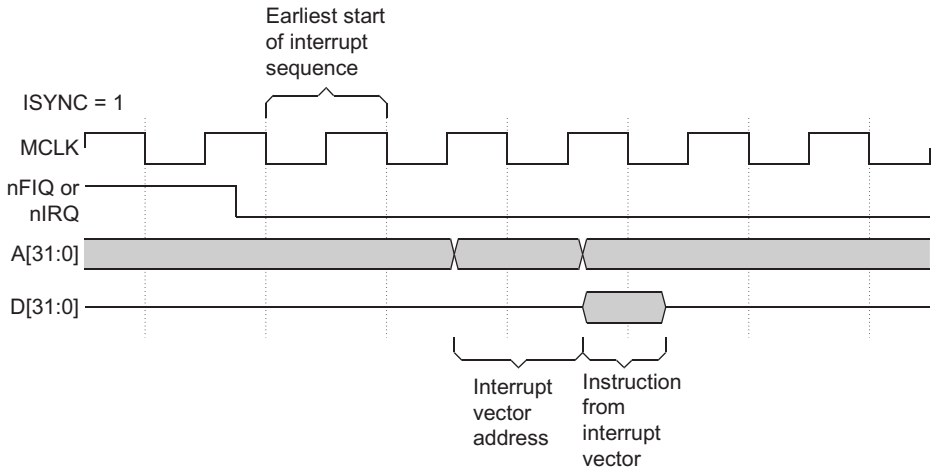


Figure 3-9 Behavior for synchronous interrupts

Asynchronous setup and hold times define timings that mean if met that the core samples the interrupt one cycle earlier than if not met. You can ignore timing violation warnings in a dynamic simulation in asynchronous interrupt mode, and you can set the relevant paths as multicycle for STA.

Figure 3-10 shows the behavior for asynchronous interrupts.

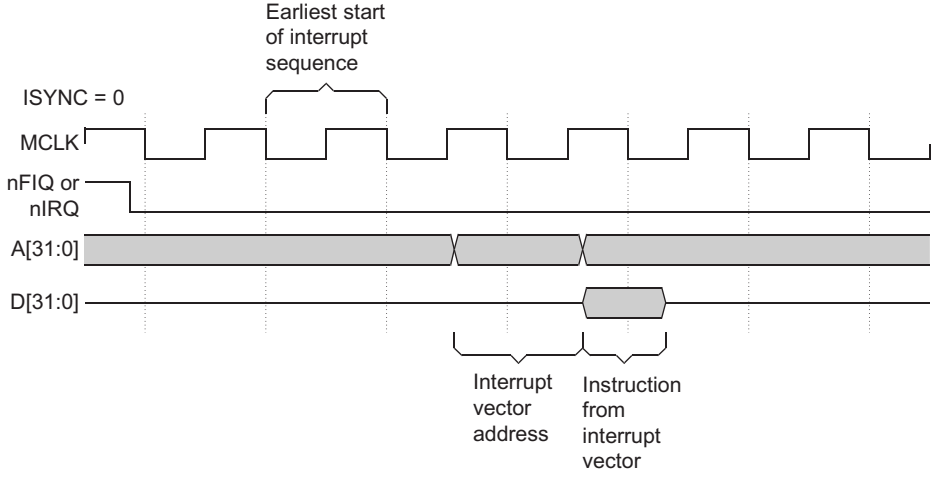


Figure 3-10 Behavior for asynchronous interrupts

Copyright © 2005 ARM Limited. All rights reserved.
Confidential

For more information on interrupt latency, see the *Interrupt Latencies* section of the *Programmer's Model* chapter in the *ARM7TDMI Rev3 Technical Reference Manual*.

Copyright © 2005 ARM Limited. All rights reserved.
Confidential

3.7 JTAG interface

This section describes how to connect the JTAG interface signals in your SoC design, see also *Debug clocks* on page 3-27.

Table 3-13 shows the JTAG interface signals present on the macrocell, and describes how you must set the signals if you intend to use JTAG debug support.

For more information on using Embedded ICE™ logic in an ARM based system, see the information on system design guidelines in the *RealView ICE User Guide*.

Table 3-13 JTAG interface signals

Signal name	Direction	Description	Connection information
nTRST	Input	Debug reset signal, active-LOW.	See *Reset* on page 3-7
TDI	Input	Scan chain data in signal.	For more information on how to connect these signals, see Figure 3-12 on page 3-27[a]
TMS	Input	Test mode select signal.	
TCK	Input	Debug logic boundary scan clock.	
TDO	Output	Scan chain data out signal.	
SDOUTBS	Input	Contains the serial data from an external scan chain.	If this signal is not required, then set the signal to logic 0
IR[3:0]	Output	TAP controller output signals[a].	If these signals are not required, then you must tie them LOW
SCREG[4:0]	Output		
TAPSM[3:0]	Output		
nTDOEN	Output	Active-LOW output pad control signal.	You must use this signal to enable tristate output pads for **TDO**, see Figure 3-12 on page 3-27
SDINBS	Output	Serial data applied to an external scan chain.	If this signal is not required, then you must leave it unconnected

a. See *Debug Support* chapter in the *ARM7TDMI Rev3 Technical Reference Manual*.

3.7.1 JTAG ID code

For the ARM7TDMI r3p0 the default IDCODE is `0x3F0F0F0F`, but this code might differ for macrocells from suppliers other than ARM Limited.

The IDCODE is tied inside the macrocell.

Copyright © 2005 ARM Limited. All rights reserved.
Confidential

3.7.2 Multiplexing JTAG ports

There is no support in RealView ICE for multiplexing **TCK**, **TMS**, **TDI**, and **TDO** between a number of different processors, therefore multiplexing JTAG ports is not recommended.

3.7.3 Daisy-chaining TAP controllers

Daisy-chaining is an extension of the JTAG board-level interconnection, and it is recommended for use with RealView ICE. Figure 3-11 shows one configuration for daisy-chaining TAP controllers in your SoC design.

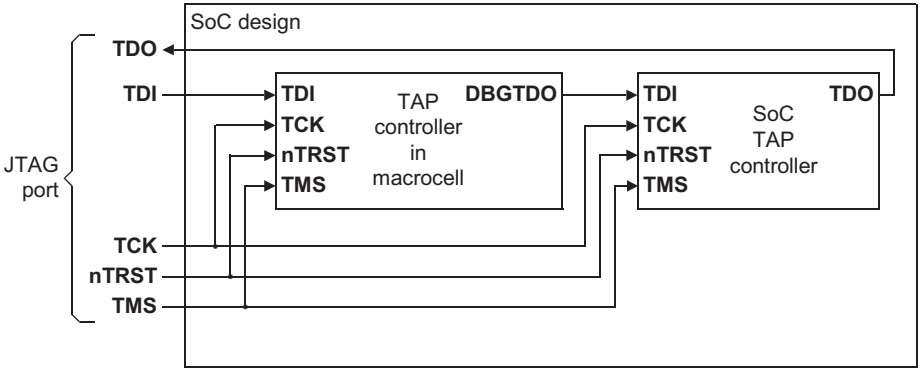


Figure 3-11 Daisy-chaining TAP controllers

Note

See Figure 3-12 on page 3-27 for more details on the debug synchronization logic.

Copyright © 2005 ARM Limited. All rights reserved.
Confidential

3.7.4 Adding scan chains to the ARM TAP controller

You can test additional logic on the SoC using the ARM TAP controller, but this is not recommended.

Note

The ARM TAP controller state machine is IEEE compliant. It is recommended that you follow the IEEE1149.1 specifications when you add scan chains to the TAP controller. However, in many cases, it is preferable to add another TAP controller for logic not supplied by ARM Limited. Some JTAG TAP controllers might provide an active-HIGH enable, for example **tdo_en**. You must invert this if the tri-state pad cell of the SoC output has an active-LOW enable input.

Copyright © 2005 ARM Limited. All rights reserved.
Confidential

3.8 Debug interface

This section describes how to connect the debug interface signals in your SoC design.

Table 3-14 shows the debug interface signals present in the macrocell, and describes how you must set the signals in your SoC design.

See the *ARM7TDMI Rev3 Technical Reference Manual* for a description of these signals.

Table 3-14 Debug interface signals

Signal name	Direction	Description	Connection information
DBGEN	Input	Debug enable signal.	**Caution** When this signal is tied off LOW debugging is not possible. If you require debug functionality, then tie this signal HIGH.
DBGRQ	Input	Debug interface extension input signals.	If you do not require these inputs, then tie them off LOW. Although not often required, the signals can be driven by additional comparators to extend the debug capabilities of the processor core.
EXTERN0	Input		
EXTERNI	Input		
BREAKPT	Input		
DBGACK	Output	Debug interface extension output signals.	These signals might be used, for example, to disable watchdog timers during debug sessions. These signals can be used by additional debug logic in the SoC, but are usually left unconnected.
RANGEOUT0	Output		You can use the **RANGEOUT** signals to trigger a logic analyzer, if your ASIC has any spare pins.
RANGEOUTI	Output		
COMMRX	Output	Debug communications channel interrupt signals.	When you require interrupt driven debug communications operation, connect these signals to the interrupt controller. If you do not require an interrupt driven debug communication channel, then you must leave these signals unconnected.
COMMTX	Output		

Copyright © 2005 ARM Limited. All rights reserved.
Confidential

3.8.1 Considerations for debug

Connection of **DBGEN** to 0 does not disable all debug functionality. It prevents breakpoints or watchpoints from being set, but it is still possible for Multi-ICE to put the core into debug state and examine registers and memory. You must not rely on **DBGEN** as a security feature.

The state **DBGEN** LOW is a power-saving feature. When you turn off the ICE logic, this reduces power consumption of the core by approximately 7%. To disable debugger access to the core, you must also disable the JTAG interface in some way, typically by forcing **nTRST** LOW, or by tying **TCK**, **TMS** or **TDI** to a fixed level within the device. This prevents any external debug access.

You can use **nTDOEN** as the **TDO** pad output enable.

3.8.2 Debug clocks

For details of the debug clocks in the macrocell see the *ARM7TDMI Rev3 Technical Reference Manual*. Figure 3-12 shows example JTAG connectivity.

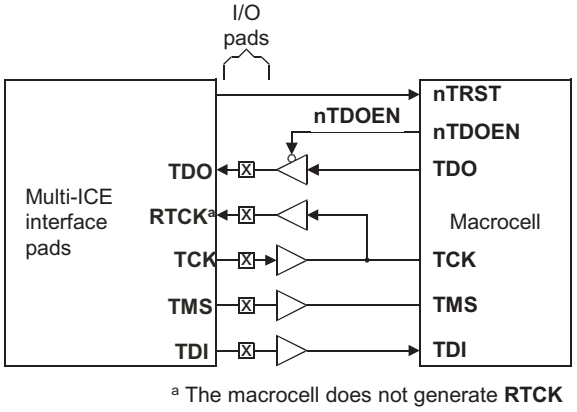


Figure 3-12 Debug connection

Note

See Figure 3-5 on page 3-8 for more details on how to generate **nTRST**.

To determine if you require a return clock, **RTCK**, at the top-level of your SoC, see the adaptive clocking and target interface logic levels information in the *System Design Guidelines* chapter of the *RealView ICE User Guide*.

Copyright © 2005 ARM Limited. All rights reserved.
Confidential

3.9 Embedded Trace Macrocell interface

If you intend to include an *Embedded Trace Macrocell* (ETM) in your SoC design, then see the information on integrating the ETM7 in the *ETM7 Technical Reference Manual*.

Caution

If you integrate ETM in your SoC design, then you must use ETM7 with the ARM7TDMI macrocell.

Note

When your design includes the *Embedded Trace Buffer* (ETB) you can only connect it to the ARM7TDMI macrocell through the AHB wrapper. The AHB-Lite port in the ETB cannot connect to the native memory interface in the ARM7TDMI macrocell. If you do not include the AHB wrapper, then you can only connect the ETB to the ETM.

The ARM7TDMI r3p0 does not have any dedicated pins for connection to ETM. To connect ETM use the native memory interface, see the *ETM7 Technical Reference Manual*. Figure 3-13 shows integration of an ETM inside the AHB wrapper.

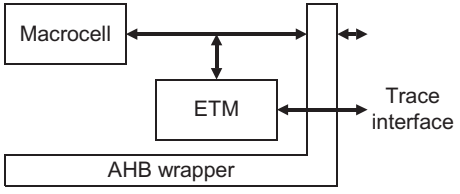


Figure 3-13 ETM integration inside AHB wrapper

Figure 3-14 shows integration of an ETM outside the AHB wrapper.

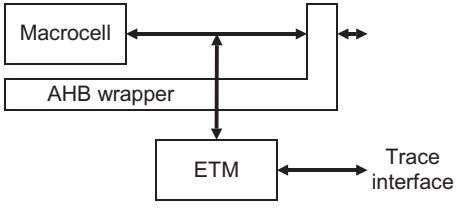


Figure 3-14 ETM integration outside AHB wrapper

Copyright © 2005 ARM Limited. All rights reserved.
Confidential

Chapter 4
DFT Guidelines

This chapter describes how to design for production testing of the macrocell. It also describes the test view of the macrocell supplied with the deliverables, the test modes, and the test interface. The chapter contains the following sections:

Copyright © 2005 ARM Limited. All rights reserved.
Confidential

4.1 About DFT guidelines

This chapter describes the various issues related to DFT that you must consider when integrating the macrocell into your SoC design.

You can use either of two methods to test the macrocell when you implement the AHB wrapper:

- test with serial vectors through the JTAG port
- test with parallel vectors through the *Test Interface Controller* (TIC) port.

Note

When you do not implement the AHB wrapper you can only use serial vectors through the JTAG port.

Table 4-1 shows the test vectors and the parts of the processor that they test.

Table 4-1 Summary of test vectors

Test name	Area	Serial vectors		Parallel vectors	
		Name	TCK cycles	Name	HCLK cycles
ale	ALE pin and address generation	arm7tdmi_ale_serr3_p1.vec	26080	arm7tdmi_aler3_p1.sim	1488
thumb	Thumb logic	arm7tdmi_thumb_serr3_p1.vec	85003	arm7tdmi_thumbr3_p0.sim	4680
arm	ARM instruction decoder	arm7tdmi_arm_serr3_p1.vec	286696	arm7tdmi_armr3_p0.sim	15696
debug	Debug features	arm7tdmi_debug_serr3_p1.vec	65717	arm7tdmi_debugr3_p0.sim	21648
ice	EmbeddedICE features	arm7tdmi_ice_serr3_p1.vec	55184	arm7tdmi_icer3_p0.sim	82272
mul	Multiplier block	arm7tdmi_mul_serr3_p1.vec	286120	arm7tdmi_mulr3_p0.sim	15642
pipe	Thumb decompression logic and pipeline	arm7tdmi_pipe_serr3_p1.vec	115340	arm7tdmi_piper3_p0.sim	6342
regnew	Register bank	arm7tdmi_regnew_serr3_p1.vec	68098	arm7tdmi_regnewr3_p0.sim	2544
-	Total cycle count	-	1018238	-	150312

Copyright © 2005 ARM Limited. All rights reserved.
Confidential

4.2 Serial test method

The serial test method requires the standard five pin JTAG interface at the SoC level. Figure 4-1 shows the JTAG interface signals for serial test vectors.

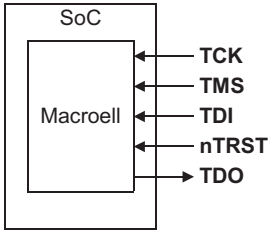


Figure 4-1 The JTAG interface for serial test

The advantage of this method is that it only requires the standard JTAG interface.

The disadvantage of this method is that it uses a relatively large number of test cycles, approximately 1 000 000.

To run the serial test vectors you must:

- provide for control and observation of the JTAG interface, see Figure 4-1
- set pins according to Table 4-2 for the duration of the tests
- ensure that there are no driver clashes between external logic and the native memory interface during serial test.

Table 4-2 Test settings for serial vectors

Signal	setting
TBE	1
MCLK	0
nRESET	1

Note

- **TBUS** is HIGH for the duration of the serial test. This means that the core drives its outputs during the test.
- The macrocell does not have an update stage on its boundary scan cells. This means that the memory bus toggles as the test signals shift in through the JTAG interface.

Copyright © 2005 ARM Limited. All rights reserved.
Confidential

To avoid the potential problems when the core interface drives outputs you can either:

- Use a top level signal that defines the values in *Test settings for serial vectors* on page 4-3 and provides control for logic connected to the core interface.
- Use the **BUSDIS** output signal to monitor the start of the serial test. This signal pulses HIGH at the start of the serial test. You can use it with a latch to control other external signals.

Copyright © 2005 ARM Limited. All rights reserved.
Confidential

4.3 Parallel test method

The parallel test method requires the AHB wrapper and a TIC block at the SoC level. Figure 4-2 shows the TIC port signals for parallel test vectors.

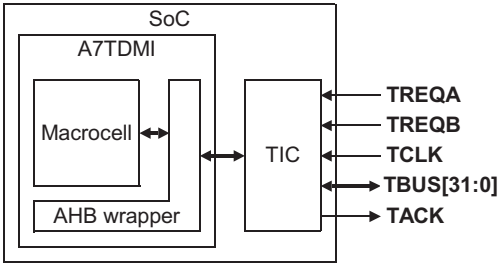


Figure 4-2 TIC test interface

The TIC port provides an interface for the parallel production vectors.

To use the parallel test method you must:

- include the AHB wrapper in your design
- provide a TIC in the SoC infrastructure
- provide for control and observation of the TIC port at the top level of the SoC.

The following sections describe how to connect the TIC and control the TIC port.

4.3.1 Connection of the TIC to the AHB wrapper

Figure 4-3 shows how the TIC block has an AHB master interface that connects to the SoC infrastructure.

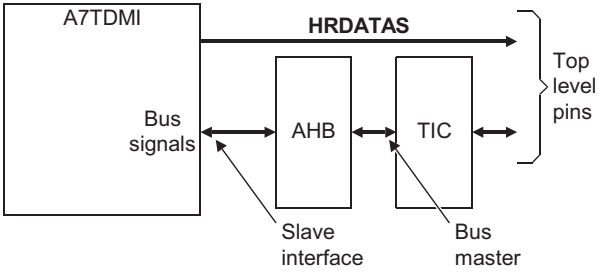


Figure 4-3 TIC connection

Copyright © 2005 ARM Limited. All rights reserved.
Confidential

4.3.2 Placement of the TIC in the address map

The parallel test vectors assume an ARM7TDMI processor slave address selected at `0xC00000000`. Because the wrapper slave interface does not have the **HADDR** input, you can place the TIC block anywhere in the address map.

4.3.3 Arbitration control of the TIC

Because the TIC is a bus master it is important that you give the TIC the highest priority for access to the AHB bus. This ensures that the TIC can access the bus when ever it requires to.

4.3.4 Control of the top level TIC test bus

The TIC test bus might share some functionality with other SoC logic. The TIC has an output, **TicRead**, that indicates when the TIC outputs data. You can use this output to ensure that there are no conflicts driving the SoC logic. Figure 4-4 shows an example system that uses **TicRead**. The connections must be duplicated for each of the 32 pads in the system.

During normal operation the functional peripheral must control the pad. During TIC testing you must ensure that the **HRDATAS** bus from the AHB wrapper can drive the pad.

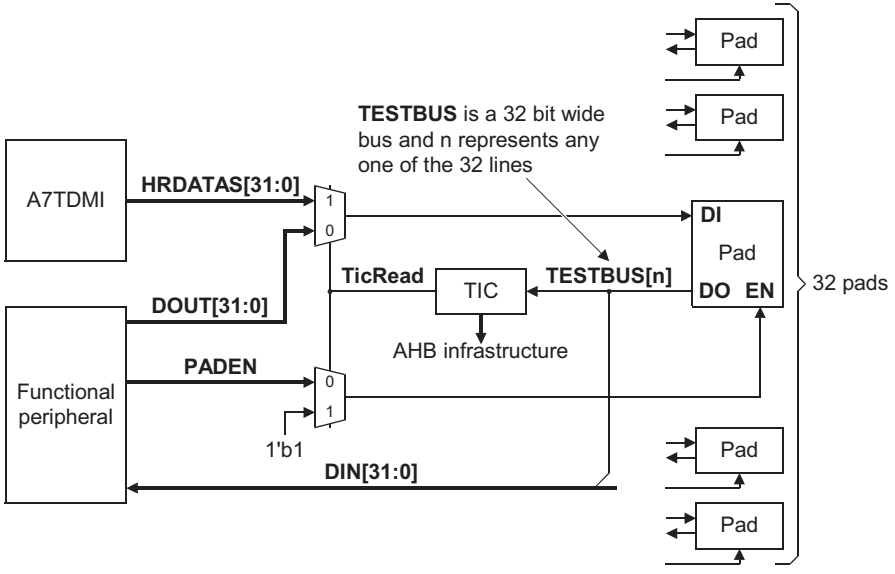


Figure 4-4 Example system control for TIC

Copyright © 2005 ARM Limited. All rights reserved.
Confidential

4.4 SoC test shadows and the macrocell

When you integrate the macrocell into your SoC, as a black-box, the internal scan chains of the macrocell are not visible to the test tools. This means that you cannot create vectors to cover any logic between the last register in the macrocell and the next register in the SoC. This invisible logic is known as a test shadow and leads to a reduction in test coverage. Figure 4-5 shows this potential test shadow.

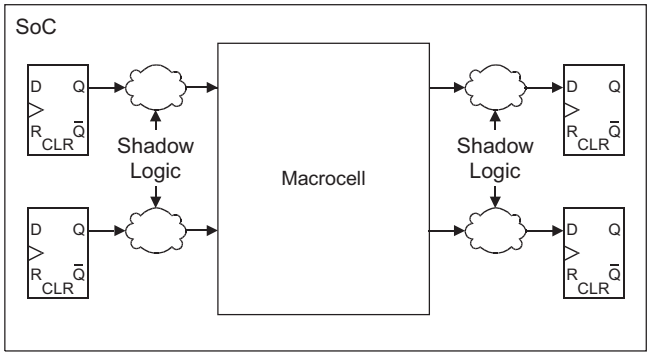


Figure 4-5 Shadow logic

One solution to this problem is to put a wrapper around the logic in the SoC to test the shadow logic. This method leaves the interconnect wires, between the wrapper and the macrocell, untested and:

- requires another wrapper placed around the macrocell
- introduces additional delay, on those signals connected to the macrocell, that might affect the overall performance of the SoC
- might introduce a minor loss of coverage between the wrapper and the macrocell.

Another solution is to perform functional testing of the design to cover the test shadow.

Copyright © 2005 ARM Limited. All rights reserved.
Confidential

Copyright © 2005 ARM Limited. All rights reserved.
Confidential

Chapter 5
Functional Verification

This chapter describes the overall functional verification process. It contains the following sections:

Copyright © 2005 ARM Limited. All rights reserved.
Confidential

5.1 About functional verification

So that you can simulate your SoC design, ARM Limited provides a DSM for the macrocell. This is a compiled model of the macrocell that is platform and simulator specific.

You can do cycle-accurate simulation of the macrocell with the DSM. The DSM supports SDF timing annotation so that you can run timing annotated simulations. For more information on timing annotation see *Dynamic simulation* on page 9-8.

This chapter describes *Using the DSM* on page 5-3.

Figure 5-1 shows the functional verification process.

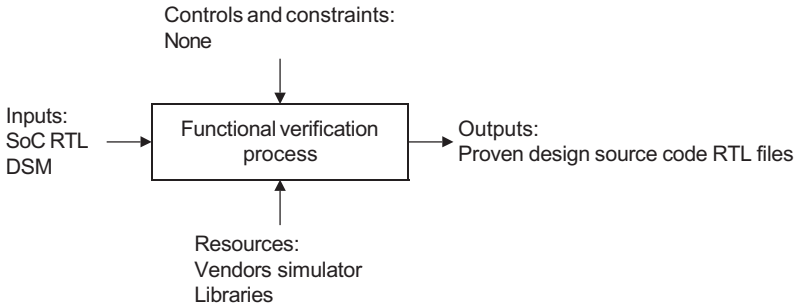


Figure 5-1 Functional verification process

Copyright © 2005 ARM Limited. All rights reserved.
Confidential

5.2 Using the DSM

This section provides information on:

- *Installing the DSM*
- *Verifying DSM installation*
- *Using the DSM for SoC simulation* on page 5-4
- *Advanced debug facilities* on page 5-4
- *Support for timing annotation* on page 5-6.

5.2.1 Installing the DSM

You must install the DSM and test it in your working environment. Because the DSM is specific to a platform and simulator, you must ensure that you have the correct DSM for your operating system and *Electronic Design Automation* (EDA) tools.

It is recommended that you follow this procedure for installation:

1. Unpack the deliverables.
2. View the DSM `README` file. This file contains instructions that you must follow for unpacking, installing, and proving the correct installation of the DSM.
3. Set the `ARM7TDMI_HOME` environment variable.
4. Source the `$ARM7TDMI_HOME/setup.cshrc` file.

Each time that you use the DSM you must set these environment variables appropriately for your system:

- `ARM7TDMI_HOME`
- `MG_LIB`
- `DIR_ARM7TDMI`.

Make a note of the appropriate settings for these variables after you have set up the DSM because you must set these correctly before you invoke the simulator.

5.2.2 Verifying DSM installation

The DSM deliverables include a verification mechanism called `CRFTESTER`. You must use `CRFTESTER` to verify the correct installation of the DSM. `CRFTESTER` also demonstrates how to use the DSM in a simulation. The `README` file describes how to use the `CRFTESTER`. The basic steps are:

1. Set all the relevant tool paths.

Copyright © 2005 ARM Limited. All rights reserved.
Confidential

2. Run the CRFTESTER. Depending on your supplied DSM structure use one of the following:
   - $ARM7TDMI_HOME/ARM7TDMICRFTESTER/crftest.csh
   - $ARM7TDMI/testing/crftest.csh
3. Check the log file for correct setup.

5.2.3 Using the DSM for SoC simulation

To use the DSM for simulation with your SoC you must:

- set the environment variables correctly, see *Installing the DSM* on page 5-3
- compile the top level Verilog wrapper, see *Compiling the Verilog wrapper*
- use the correct simulation options for the Verilog PLI, see *Setting the Verilog PLI options*.

Compiling the Verilog wrapper

You must compile the top level Verilog wrapper for your simulation library. To do this compile the file:

$ARM7TDMI_HOME/ARM7TDMI/ARM7TDMI.v

You must include the compiled file in your Verilog simulation library so that the Verilog module ARM7TDMI appears in your work library.

For full details see the README file.

Setting the Verilog PLI options

The Verilog PLI options vary depending on the target simulator. The CRFTESTER provides an example that shows how to invoke the PLI options.

5.2.4 Advanced debug facilities

The DSM provides two advanced debug features, see:

- *Programmers' Model view for debug* on page 5-5
- *EIS tracing* on page 5-5.

Copyright © 2005 ARM Limited. All rights reserved.
Confidential

Programmers' Model view for debug

To aid debug during functional simulation of your design, the DSM provides additional signals, present within the Verilog wrapper, that export the programmers' model. Figure 5-2 shows the hierarchy.

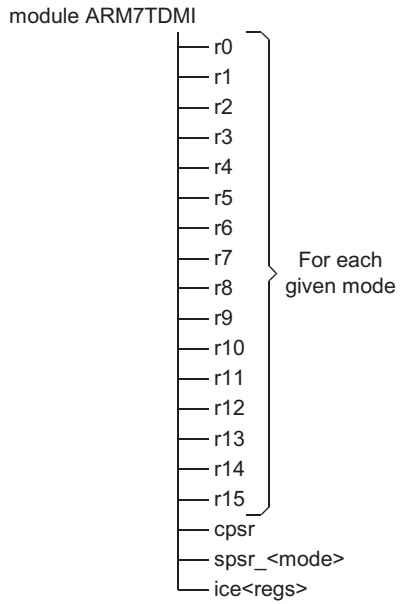


Figure 5-2 Programmers' model in the Verilog wrapper

For example, suppose the design in your testbench is `uMySoc` and the instance of the macrocell is `uCore`. When you trace `uMySoc/uCore/r15` this generates trace for r15, the program counter, in the core. When the trace increments this indicates that the core is fetching instructions and executing them.

EIS tracing

Execution Instruction Stream (EIS) provides a means to trace and view all the instructions that pass through the macrocell. You can create an output text file that contains trace of all the instructions that pass through the core.

To enable EIS trace set the environment variable:

```
setenv ARM7TDMI_DISASS
```

With this variable set the DSM sends the EIS log to STDOUT. To change the output to a file:

Copyright © 2005 ARM Limited. All rights reserved.
Confidential

```
setenv ARM7TDMI_DISASS_OUTPUT <filename>
```

Where <filename> might, for example, be log.eis.

5.2.5 Support for timing annotation

The DSM supports annotation of SDF timing information, see *Dynamic simulation* on page 9-8.

Copyright © 2005 ARM Limited. All rights reserved.
Confidential

Chapter 6
DFT Verification

This chapter describes the DFT verification of the macrocell. It contains the following sections:

Copyright © 2005 ARM Limited. All rights reserved.
Confidential

6.1 About DFT verification

This chapter describes issues related to verification of the DFT integration of the macrocell and the capture and replay of test vectors. The macrocell supports functional test techniques that apply either:

- serial test vectors through the JTAG port
- AMBA SIM vectors through the AHB wrapper.

This chapter describes how to:

- use the supplied JTAG vectors to test the macrocell
- use the supplied SIM vectors to test the macrocell
- test external logic connected outside the macrocell.

Note

- The preferred test method is with the AMBA SIM vectors. If you do not have the AHB wrapper, then you must use the serial test vectors.
- All test vectors use the DSM for replay.

6.1.1 DSM warnings

The DSM might generate the following warnings, during replay of vectors, that the simulator shows in the console:

```
** Warning: Illegal MUX setup in PSR
** Warning: Illegal SPSR write [MODE &0C]
** Warning: Illegal SPSR read [MOde &0C]
** Warning: Illegal register select (-1->B) in RegBank
** Warning: Illegal register select (-1->A) in RegBank
** Warning: Illegal SPSR write [Mode &0F]
```

You can ignore these warnings because they occur when the test vectors apply non-standard test patterns to exercise paths in the core efficiently.

6.1.2 Vector output comparison

The vectors supplied with the macrocell do not check output pins at certain points, for example when the core is reset. These points are marked in the vectors as don't care, X, states. It is important to preserve these states during final vector capture. If you do not preserve these states, then you might observe failures in the final tester.

Copyright © 2005 ARM Limited. All rights reserved.
Confidential

6.2 Using the supplied JTAG serial vectors

The macrocell is supplied with JTAG serial vectors that provide high test coverage of the macrocell. When you integrate the macrocell in your SoC, you must ensure that you make the JTAG test pins available at the top level in test mode, see *Serial test method* on page 4-3.

Note

If you do not make the pins available, then you can not replay the serial vectors.

The JTAG serial vectors are available in these formats:

- Verilog
- *Wave Generation Language* (WGL).

The recommended order for use of the vectors from the top level of the SoC design is:

1. Replay at the macrocell level. This checks that you have correctly installed the vectors.
2. Replay at the SoC level, see *Replay of serial vectors at the SoC level* on page 6-4.

6.2.1 Replay of serial vectors at the macrocell level

The first stage for replay of the serial vectors is to replay them at the macrocell level. This checks that the vector and DSM installation are correct.

To replay the vectors, use a simulator that is compatible with the DSM to compile your version of the Verilog testbench. You must also compile the top level Verilog wrapper for the DSM, `ARM7TDMI.v`. Figure 6-1 shows how the environment corresponds for replay.

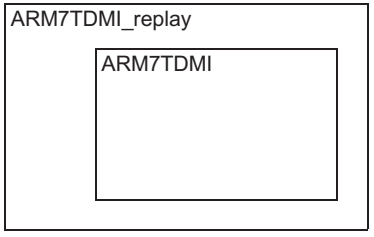


Figure 6-1 Replay environment for serial vectors at the macrocell level

When you run simulations with the vectors they automatically terminate the simulation.

Copyright © 2005 ARM Limited. All rights reserved.
Confidential

When your test vector replay is successful the simulator displays the message:

```
# TEST COMPLETED WITH NO ERRORS
```

Note

- You might have to convert the WGL files to a format suitable for the tester that your chosen manufacturer uses to test the production silicon.
- To simulate the WGL vectors you must convert them into a more suitable format.

  For example, convert the WGL vectors into a Verilog simulation testbench using the VTRAN tool from Source III Inc.

  Because the WGL format contains the same patterns as those in the Verilog testbench, it is not necessary to replay these at the macrocell level.
- You can use any tool that accepts WGL format and writes out a Verilog testbench.

6.2.2 Replay of serial vectors at the SoC level

To replay the serial vectors at the SoC level you must modify the pin names defined in the vector to match your design. For example, suppose your SoC calls the JTAG pins:

- **ARMTDI**
- **ARMTMS**
- **ARMnTRST**
- **ARMTCK**
- **ARMTDO**.

In this case you modify the Verilog vector to replace the pins, for example **TDI**, with those in your SoC.

Copyright © 2005 ARM Limited. All rights reserved.
Confidential

6.3 Using the supplied AMBA TIC vectors

The macrocell is supplied with AMBA test vectors that provide high test coverage of the macrocell. You must have the AHB wrapper to use these vectors, see *Parallel test method* on page 4-5. When you integrate the macrocell in your SoC, you must ensure that you make the TIC pins available at the top level in test mode, see Figure 6-2.

Note

If you do not make the pins available, then you can not replay the vectors.

ARM Limited only supply the AMBA vectors in SIM format and you must replay and capture the vectors on your design with your chosen vector format, for example VCD.

The recommended method for vector replay is to create a testbench that includes a behavioral model called a TicBox that ARM Limited supply. Figure 6-2 shows how you can use this testbench to replay the vectors.

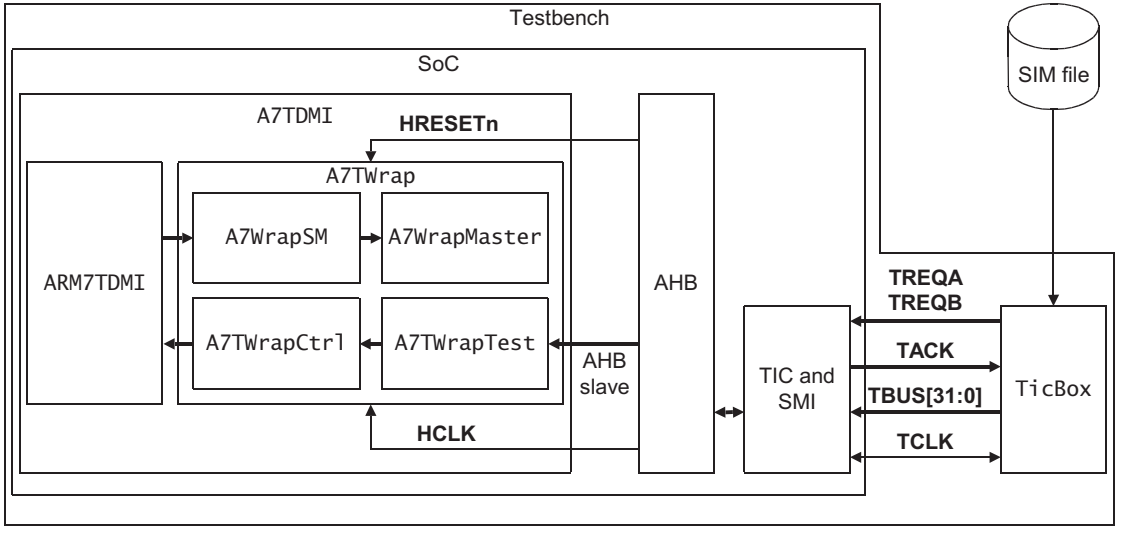


Figure 6-2 AMBA test vector replay

Copyright © 2005 ARM Limited. All rights reserved.
Confidential

6.3.1 Replay of TIC vectors

To replay the TIC vectors, use a simulator that is compatible with the DSM to compile your version of the Verilog testbench. Run the simulation and capture the vectors for these pins:

- **TREQA**
- **TREQB**
- **TACK**
- **TBUS**
- **TCLK**.

The TicBox model reads the file `infile.sim`. When the replay is successful the Verilog console does not show any errors.

6.3.2 Capturing TIC vectors

You must capture TIC vectors in your chosen format, for example VCD.

Copyright © 2005 ARM Limited. All rights reserved.
Confidential

Chapter 7
Design Synthesis

This chapter describes, in outline, what you must do to synthesize the macrocell for integration into your design. It contains the following sections:

Copyright © 2005 ARM Limited. All rights reserved.
Confidential

7.1 About synthesis

This chapter describes the synthesis stages of your SoC design with the macrocell. The chapter assumes that you are familiar with synthesis procedures and only describes the steps required to use the macrocell timing views. This chapter does not describe any specific methodology, such as top-down or bottom-up.

Figure 7-1 shows the synthesis process.

Note

All commands and syntax are for Synopsys synthesis tools and are given in *Tool Command Language* (TCL). For other tools you must derive the equivalent commands.

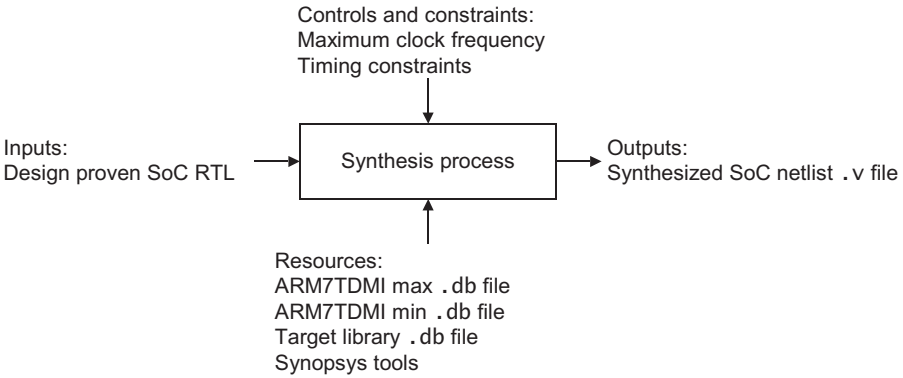


Figure 7-1 Synthesis process

Copyright © 2005 ARM Limited. All rights reserved.
Confidential

7.2 Timing models

You are provided with timing views of the macrocell in the Liberty format, `.lib` files, and precompiled versions of the `.lib` files in `.db` format. You can use these with either the Synopsys synthesis and place and route or other synthesis and place and route tools. Alternative place and route tools might require conversion from the `.lib` format. The timing views are context independent and contain all the necessary input and output requirements for the macrocell.

It is recommended that you compile the Liberty source `.lib` files to `.db` files using your preferred version of the Synposys tools.

Use the `.lib` files for any other tools that require a timing view of the macrocell.

Table 7-1 shows the macrocell `.lib` files. In this table `<process>` depends on the specific foundry technology.

Table 7-1 Summary of synthesis deliverables

Filename	Description	Notes
`ARM7TDMI_<process>_ff2_min.lib`	Liberty `.lib` fast timing model for best-case conditions V= V_{dd} +10% T = temperature[a]	Liberty library source is provided for use with other tools, for example layout and place and route. Use these files for given conditions in the Synopsys synthesis flow.
`ARM7TDMI_<process>_ss_max.lib`	Liberty `.lib` slow timing model for worst-case conditions V= V_{dd} -10% T = temperature[a]	

a. See the *ARM7TDMI Core Configuration and Performance Summary* document.

7.2.1 Using the timing models

This section shows how to alter your `dc_shell` setup for use with the timing model.

To use the supplied macrocell timing models in your Synopsys environment, you must alter your existing `dc_shell` setup:

1. Add the installation location of the synthesis deliverables to the variable `search_path`:

```
set ARM7TDMI_lib_loc "full path where the ARM7TDMI library files are located"
set search_path    [concat $search_path $ARM7_lib_loc]
```

2. Add the macrocell timing files into your link library:

```
set ARM7_slow_lib ARM7TDMI_<process>_ss_max.db
```


Copyright © 2005 ARM Limited. All rights reserved.
Confidential

```
set ARM7_fast_lib ARM7TDMI_<process>_ff2_min.db
set link_path       [concat * $ARM7_slow_lib]
```

3. Set the correct library conditions:

```
set_min_library $ARM7_slow_lib -min_version $ARM7_fast_lib
```


Copyright © 2005 ARM Limited. All rights reserved.
Confidential

Chapter 8
Physical Integration Guidelines

This chapter describes the physical integration of one or more instances of the macrocell in a SoC. It contains the following sections:

Copyright © 2005 ARM Limited. All rights reserved.
Confidential

8.1 About physical integration

You must perform a number of different checks and tasks during the physical integration stage of the design flow. These checks and tasks are described in more detail later in the chapter. Figure 8-1 shows the physical integration process. Physical integration involves several steps:

- floorplanning
- place and route
- physical integration checks.

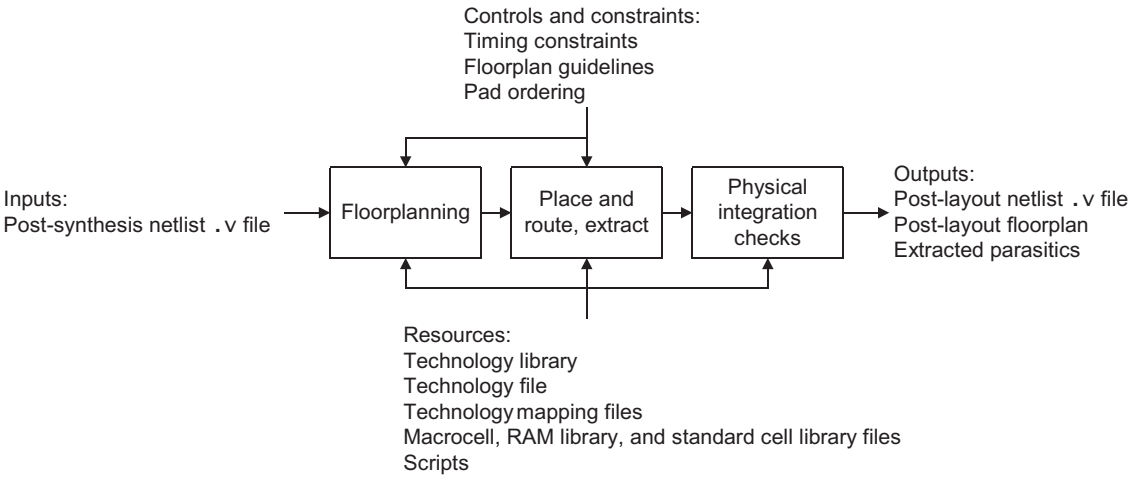


Figure 8-1 Physical integration process

Note

The tasks mentioned highlight the issues, related to the macrocell, to consider during physical integration. It is not an exhaustive list of tasks, and you must have an in-depth knowledge of place and route to complete physical integration.

Copyright © 2005 ARM Limited. All rights reserved.
Confidential

8.2 Floorplanning

This section provides information on physical layout of the macrocell and manufacturing considerations for:

- *Placement*
- *Orientation* on page 8-6
- *Power connections* on page 8-6
- *Signal connections* on page 8-7.

8.2.1 Placement

Figure 8-2 on page 8-4 shows the pin positions for the left hand side of the macrocell and Figure 8-3 on page 8-5 shows the pin positions for the right hand side of the macrocell. The address bus **A[31:0]** is brought out on two sides of the core to give easier floor-planning of the layout around the macrocell. You can connect to it from only one side.

Copyright © 2005 ARM Limited. All rights reserved.
Confidential

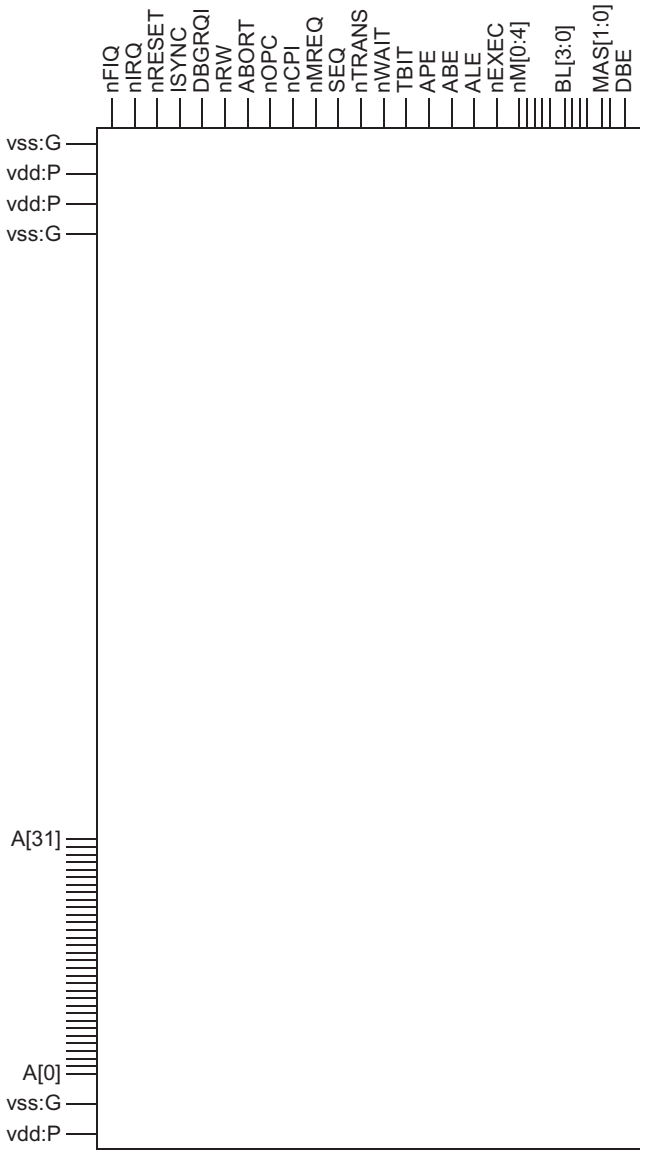


Figure 8-2 Left hand side pin positions for macrocell

Copyright © 2005 ARM Limited. All rights reserved.
Confidential

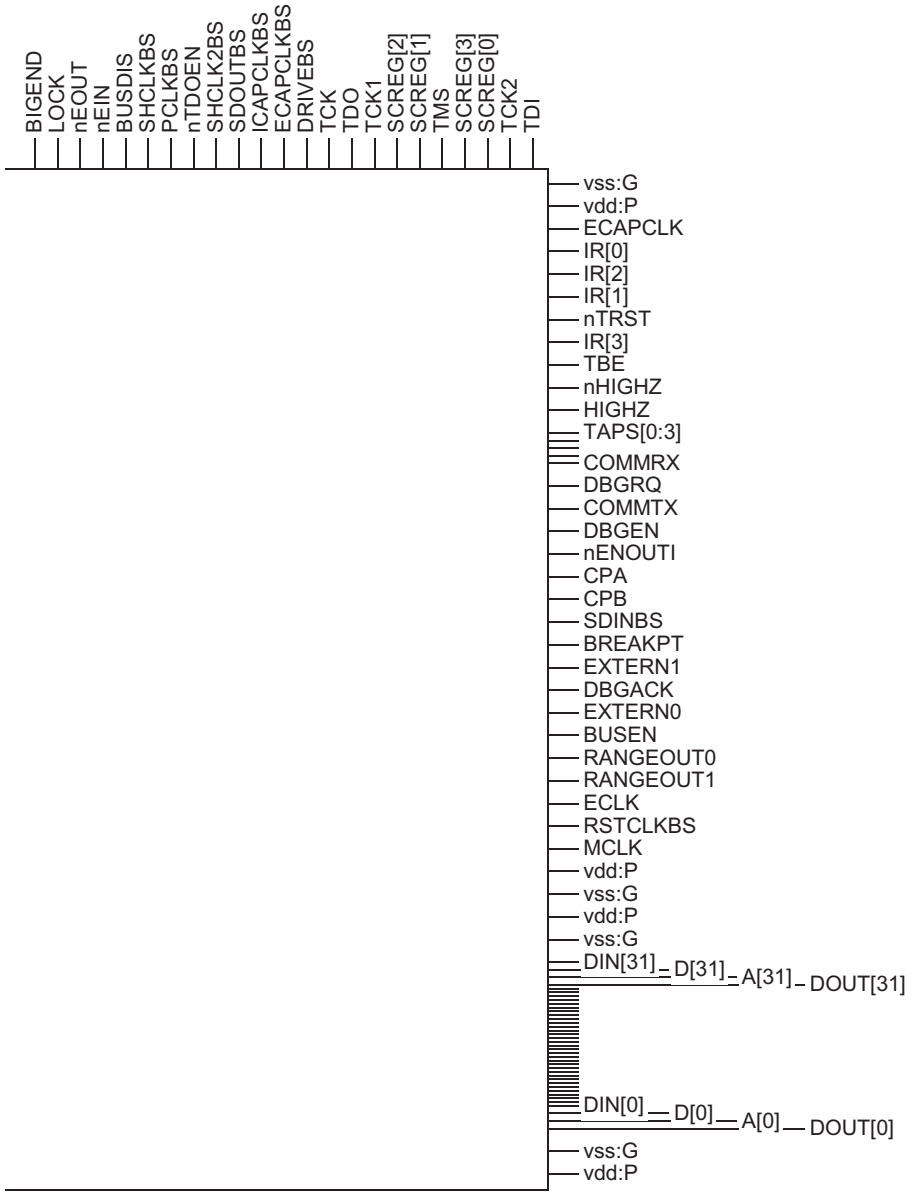


Figure 8-3 Right hand side pin positions for macrocell

Copyright © 2005 ARM Limited. All rights reserved.
Confidential

You must decide where to place the macrocell, for example in the corner or the centre of the SoC, and the implications of your choice. The implications might be:

- Access to the pads of the SoC around the macrocell might be difficult because of routing restrictions. See *Routing restrictions* on page 8-9.
- You might require routing channels with soft blockages around the edge of the macrocell.
- Timing of pads blocked by the macrocell might be severely affected, therefore pay special attention when you allocate interfaces for blocked pads. Where possible, allocate these blocked pads to interfaces that are not speed-critical.

Note

You must ensure that the placement of the macrocell is so that it snaps to the grid. Failure to do this results in off grid routing problems when connecting to the macrocell pins.

8.2.2 Orientation

It is recommended that you integrate the macrocell so that the orientation suits the preferred routing direction and that you choose either 0°, 180° or mirror placements from the natural orientation of this block. Appropriate orientation prevents problems in routing to pins in the non-preferred routing direction.

Note

You can rotate the macrocell by 90°. If you do this, then the connections to some of the pins on the macrocell might be in the non-preferred routing direction. This can lead to routing congestion in these areas.

8.2.3 Power connections

You must connect all power lines, V_{SS} and V_{DD}, to the macrocell in a width at least the size of the V_{SS} and V_{DD} pins. Ideally, continue this structure outside the macrocell in the SoC. You can alter the width and pitch of routes, to match the grid area of the standard cell, as long as an appropriate amount of power reaches each macrocell boundary. See the *ARM7TDMI Core Configuration and Performance Summary* document to determine on what layers the macrocell power pins are available.

Copyright © 2005 ARM Limited. All rights reserved.
Confidential

Note

- The macrocell has been analyzed assuming ideal voltage and current sources at all macrocell power pins. You must ensure that sufficient voltage and current is supplied to all the macrocell power pins.
- You must not use the macrocell as a pass-through to supply power to other blocks in the SoC.

8.2.4 Signal connections

The macrocell pins must not be obscured by the corner of the SoC so the pins must be adjacent, and close, to a standard cell region. Figure 8-4 shows the correct way to place the signals.

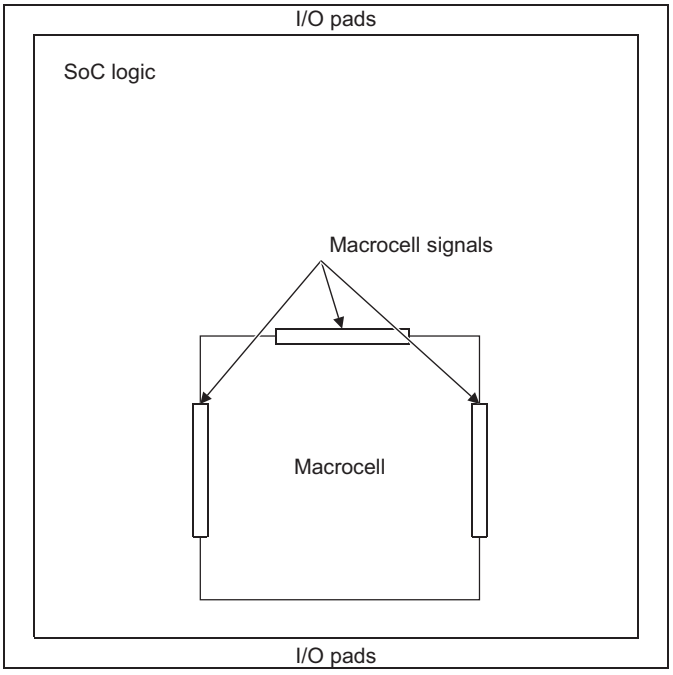


Figure 8-4 Correct signal connections

Figure 8-5 on page 8-8 shows an example of poor placement of the macrocell. This placement makes it difficult to route the macrocell pins to the standard cell logic. The result is a severe reduction in the speed of the macrocell interfaces.

Copyright © 2005 ARM Limited. All rights reserved.
Confidential

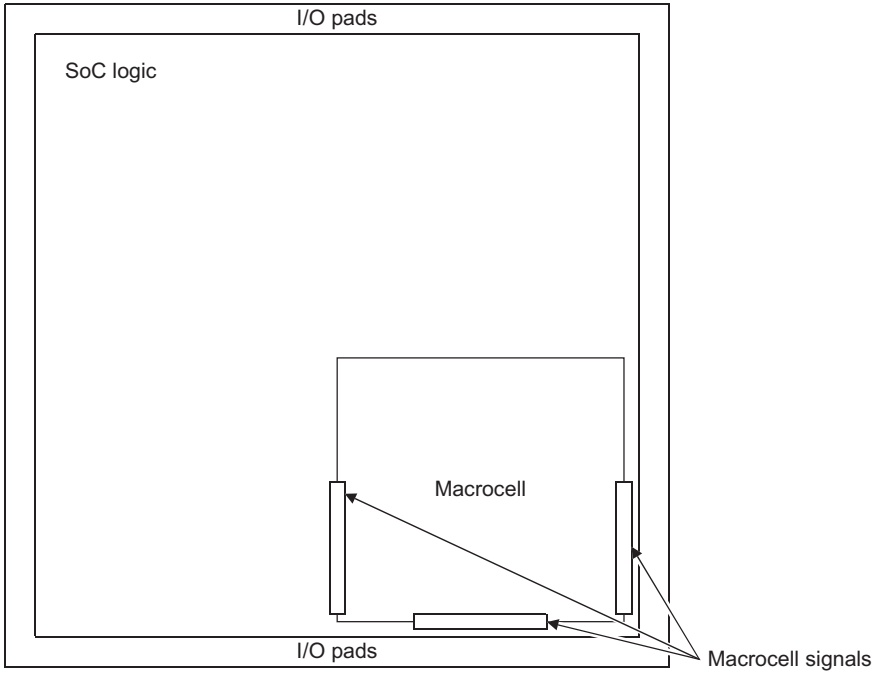


Figure 8-5 Bad signal connections

Copyright © 2005 ARM Limited. All rights reserved.
Confidential

8.3 Place and route

When you have a floorplan for your SoC design you can start place and route. See *Floorplanning* on page 8-3 for information on floorplanning. This section has the following parts:

- *Routing restrictions*
- *Antennas*.

8.3.1 Routing restrictions

Routing over the macrocell might impact performance because of the capacitive effects of adjacent metal layer wiring. Therefore, certain routing restrictions apply to the macrocell depending on the process you use. These restrictions depend on the number of metal layers used within the macrocell and the number of layers available in your choice of process.

Power and signal pins are accessible on single layers when integrating the macrocell into your SoC. It is recommended that you give careful consideration to the metal layers used for supply and signal routing, to maintain good quality results and routability.

See the *ARM7TDMI Core Configuration and Performance Summary* datasheet for more details on the number of metal layers used and the routing restrictions that result.

8.3.2 Antennas

The macrocell is verified and passes antenna checks for the target manufacturing process.

Antenna repair

To assist antenna repair at the SoC level, ARM Limited provides technology files for routing tools that support antenna repair. These technology files are in the macrocell CLF or LEF antenna views. You are strongly recommended to use either of these files during the place and route stage.

Antenna checking

After stream out of the SoC containing the macrocell you are likely to find antenna rule violations when you use tools for transistor level checking, such as Mentor Graphics Calibre. These rule violations appear because of the different ways in which EDA tools calculate antenna ratios.

Copyright © 2005 ARM Limited. All rights reserved.
Confidential

Use the antenna GDS2 view provided with the macrocell in place of the normal footprint GDS2 view for final antenna checking. The antenna GDS2 view is not DRC clean, but replicates the antenna view of the macrocell.

When you have produced an antenna clean layout with the antenna GDS2 view in the transistor level checks, remove antenna GDS2 view and put the normal footprint view of the macrocell back in.

For an example flow see *Design Rule Check* on page 8-11.

Antenna avoidance

To avoid antennas you are recommended to insert a jog to the top routing layer near each of the macrocell pins.

Copyright © 2005 ARM Limited. All rights reserved.
Confidential

8.4 Physical integration checks

This section describes the checks that you must perform as part of the physical integration process:

- *Layout Versus Schematic*
- *Design Rule Check*
- *Antenna checks* on page 8-12.

8.4.1 Layout Versus Schematic

Layout Versus Schematic (LVS) verifies the connectivity of the layout with that of the schematic. You can run LVS at either:

- the gate or cell level
- the transistor level.

Because the macrocell is obscured, you must verify that the intended connections to the macrocell are correct by doing these checks in this order:

1. Cell level LVS.
2. Transistor level LVS.

Note

LVS does not check that all the power and ground pins of the macrocell are connected. Therefore, you must check that all power and ground pins on the macrocell are connected.

8.4.2 Design Rule Check

The *Design Rule Check* (DRC) verifies that the SoC design can be manufactured according to the physical design rules from the foundry.

To perform DRC correctly, you must use the information supplied in the abstracts for the macrocell. Figure 8-6 on page 8-12 shows an example flow for DRC.

Copyright © 2005 ARM Limited. All rights reserved.
Confidential

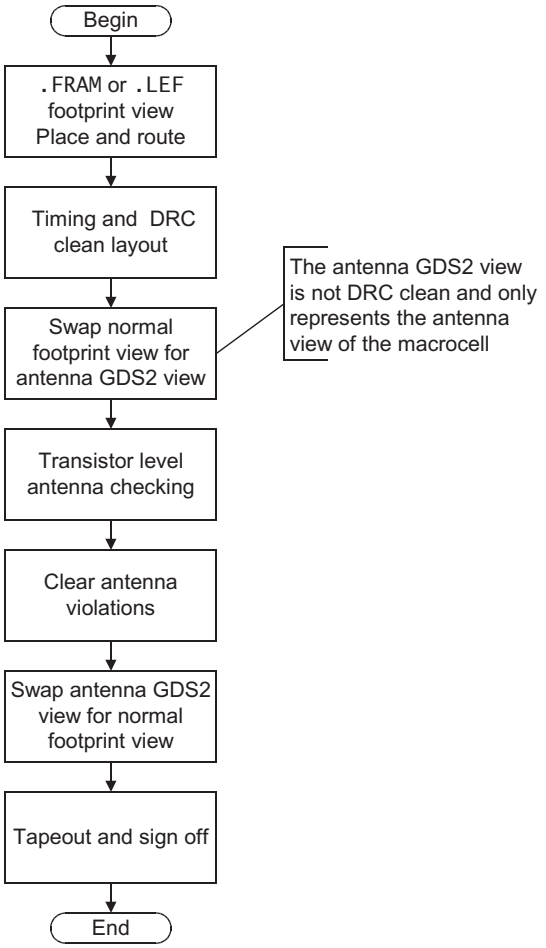


Figure 8-6 Flow for DRC

8.4.3 Antenna checks

Antenna checks verify that the metal layers conform to a particular manufacturing check. These checks are occasionally included with the DRC.

To ensure that there are no antenna rule violations that involve the macrocell, you must use the preventative measures mentioned in *Antennas* on page 8-9. You might also choose to have a full SoC antenna check at the foundry during the core merge process.

Copyright © 2005 ARM Limited. All rights reserved.
Confidential

Chapter 9
Post-Layout Verification

This chapter describes post-synthesis and post-layout netlist timing and functional verification of your SoC design. It contains the following sections:

Copyright © 2005 ARM Limited. All rights reserved.
Confidential

9.1 About verification

The two types of verification you can do on the macrocell are:

- static verification
- dynamic verification.

Static verification includes *Logical Equivalence Checking* (LEC) and STA. Dynamic verification usually involves simulation of a back annotated, post-layout netlist.

You are recommended to perform these verification steps:

- use LEC to prove that the RTL and post-synthesis netlist are logically equivalent
- use LEC to prove that the post-synthesis netlist and post-layout netlist are logically equivalent
- use STA to prove that the post-layout netlist meets the timing goals for your SoC
- perform netlist simulations on a back annotated post-layout netlist.

Figure 9-1 shows the LEC verification process. The LEC tool might be, for example, Synopsys Formality.

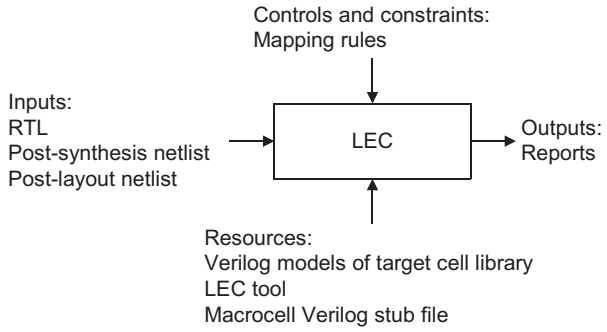


Figure 9-1 LEC verification process

Figure 9-2 on page 9-3 shows the STA verification process.

Copyright © 2005 ARM Limited. All rights reserved.
Confidential

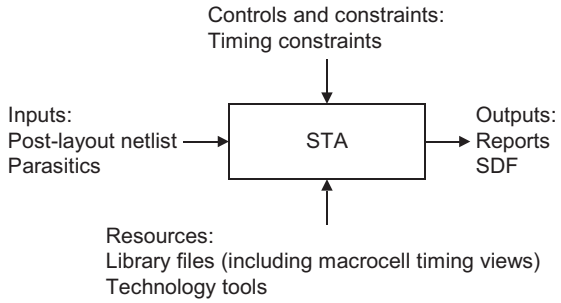


Figure 9-2 STA verification process

Figure 9-3 shows the dynamic verification process.

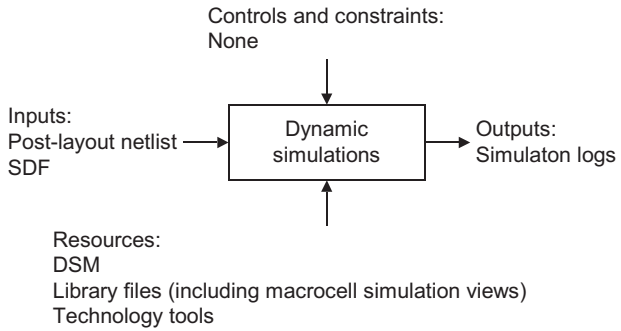


Figure 9-3 Dynamic verification process

Copyright © 2005 ARM Limited. All rights reserved.
Confidential

9.2 Logical Equivalence Checking

To equivalence check SoC netlists that include the macrocell, at either the post-synthesis or the post-layout stages, you must instruct the LEC tool to obscure, or black box, the macrocell. This must happen in all stages when you are performing:

- an RTL to post-synthesis netlist comparison of your SoC design
- a post-synthesis to post-layout netlist comparison of your SoC design
- a post-layout to *Engineering Change Order* (ECO) of your post-layout netlist comparison of your SoC design.

9.2.1 Use of a Verilog stub file

To successfully obscure the macrocell, you must read in a Verilog stub file for the macrocell. This file enables the logical equivalence checker to:

- turn the macrocell outputs into extra SoC inputs that can be set to any value when driving SoC logic
- turn the macrocell inputs into extra SoC outputs, to make the inputs part of the equivalence check.

To create the Verilog stub file, extract the top level module and I/O declarations from the `ARM7TDMI.v` file supplied with the DSM.

Copyright © 2005 ARM Limited. All rights reserved.
Confidential

9.3 Static Timing Analysis

This section provides information on how to perform STA on your post-layout SoC design with the macrocell in the following sections:

- *Supplied timing views*
- *Performing STA* on page 9-6.

9.3.1 Supplied timing views

The delay times given in the `max` library are the longest possible time it can take for the signal to change, from the clock edge. You can use this to check propagation delays and setup times. The delay time given in the `min` library is the shortest possible time it can take for the signal to change. This is effectively an output hold time, that is how long after the clock edge the old value can be considered valid. You can use this to check for hold time problems.

In some cases there is no delay time, because the output hold time is not characterized as part of the characterization process before fabrication. Table 9-1 shows a summary of the macrocell timing files.

Table 9-1 Summary of.lib timing files

Filename	Library Name	Description
`ARM7TDMI_<process>_ff1_max.lib`	`ARM7TDMI`	Liberty `.lib` fast timing model for best case conditions, slowest paths V = Vdd+10% T = temperature[a]
`ARM7TDMI_<process>_ff1_min.lib`	`ARM7TDMI`	Liberty `.lib` fast timing model for best case conditions, fastest paths V = Vdd+10% T = temperature[a]
`ARM7TDMI_<process>_ff2_max.lib`	`ARM7TDMI`	Liberty `.lib` fast timing model for best case conditions, slowest paths V = Vdd+10% T = temperature[b]
`ARM7TDMI_<process>_ff2_min.lib`	`ARM7TDMI`	Liberty `.lib` fast timing model for best case conditions, fastest paths V = Vdd+10% T = temperature[b]
`ARM7TDMI_<process>_ss_max.lib`	`ARM7TDMI`	Liberty `.lib` slow timing model for worst case conditions, slowest paths V = Vdd-10% T = temperature[c]

Copyright © 2005 ARM Limited. All rights reserved.
Confidential

Table 9-1 Summary of.lib timing files (continued)

Filename	Library Name	Description
ARM7TDMI_<process>_ss_min.lib	ARM7TDMI	Liberty .lib slow timing model for worst case conditions, fastest paths V = Vdd-10% T = temperature[c]
ARM7TDMI_<process>_tt_max.lib	ARM7TDMI	Liberty .lib typical timing model for typical case conditions, slowest paths V = Vdd T = temperature[c]
ARM7TDMI_<process>_tt_min.lib	ARM7TDMI	Liberty .lib typical timing model for typical case conditions, fastest paths V = Vdd T = temperature[c]

a. See the *Core Configuration and Performance Summary*.
b. Different temperature from[a], see the *Core Configuration and Performance Summary*.
c. See the *Core Configuration and Performance Summary*.

9.3.2 Performing STA

This section describes how to perform STA on the macrocell.

Note

- The commands in this section are in TCL and assume that:
  - you use Synopsys PrimeTime for STA
  - the .lib files, described in *Timing models* on page 7-3, are already compiled to .db files.
- This section is for information only and is provided as an example for single best and worst case STA. You must also use these guidelines in the relevant part of the STA scripts. This section gives guidelines on how to set up the tool to use the timing models of the macrocell for STA.

To perform STA you must:

1. Ensure that the compiled .lib files of the macrocell are in your Synopsys path. The following must be present in your PrimeTime setup path:

```
set ARM7_lib_loc "pull path where the ARM7TDMI library files are located"
set ARM7_slow_lib ARM7TDMI_<maxdelay>.db
set ARM7_fast_lib ARM7TDMI_<mindelay>.db
set search_path    [concat $search_path $ARM7_lib_loc]
```


Copyright © 2005 ARM Limited. All rights reserved.
Confidential

```
set link_path       [concat * $ARM7_slow_lib]
```

2. Establish a maximum and minimum library relationship:

   ```
   set_min_library $ARM7_slow_lib -min_version $ARM7_fast_lib
   ```

3. Set the operating conditions:

   ```
   #
   # The timing models of the ARM7TDMI hardened macrocell
   # does not have specific named operating conditions, so a
   # symbolic operating condition name is created for
   # these libraries using the create_operating_conditions command
   # This is necessary if you want to do min-max analysis
   #
   # EXAMPLE ONLY! For a process where the temperature for
   # the fast corner is 0C.
   #
   create_operating_condition \
   -name fast \
   -library ARM7TDMI_<min>.db:ARM7TDMI_<min> \
   -process 1.00 \
   -temperature 0.00 \
   -voltage 1.98
   create_operating_condition \
   -name slow \
   -library ARM7TDMI_<max>.db:ARM7TDMI_<max> \
   -process 1.00 \
   -temperature 125.00 \
   -voltage 1.62
   #
   # Now set the operating condition using the symbolic names
   #
   set_operating_condition -analysis_type bc_wc \
   -min fast \
   -min_library $ARM7_fast_lib:ARM7TDMI \
   -max slow \
   -max_library $ARM7_slow_lib:ARM7TDMI
   ```

4. Deal with the clocks:

   You must create the clocks for your SoC and ensure that they are declared as propagated.

Copyright © 2005 ARM Limited. All rights reserved.
Confidential

9.4 Dynamic verification

This section provides information on how to perform *Dynamic simulation*.

9.4.1 Dynamic simulation

You can simulate the behavior of the netlist of the post-layout SoC containing the macrocell. To do this use the DSM of the macrocell. You must perform the simulation with timing annotation. The DSM supports *Standard Delay Format* (SDF) timing annotation using the conventional SDF annotation method, for example, in Verilog:

```
$sdf_annotate
```

You can generate SDF information for the complete design, for example with the write-sdf command in Synopsys tools. This SDF only contains boundary timing information for the ARM7TDMI macrocell, similar to other library components such as SRAM. To annotate the SDF on to the DSM, you must post process the SDF with the `sdfremap` tool that is supplied with the DSM. This tool reformats the SDF entries for the macrocell I/O in a suitable form for annotation. If you do not perform this step correctly, then you cannot achieve complete timing annotation.

Timing shell annotation

Figure 9-4 on page 9-9 shows the hierarchy of the DSM for the SDF timing annotation. Here the top level Verilog DSM, `ARM7TDMI.v`, defines the ARM7TDMI module but the timing annotation must occur at the `ARM7TDMI_timing` level. The `sdfremap` tool automatically annotates timing at the correct level in the hierarchy.

Copyright © 2005 ARM Limited. All rights reserved.
Confidential

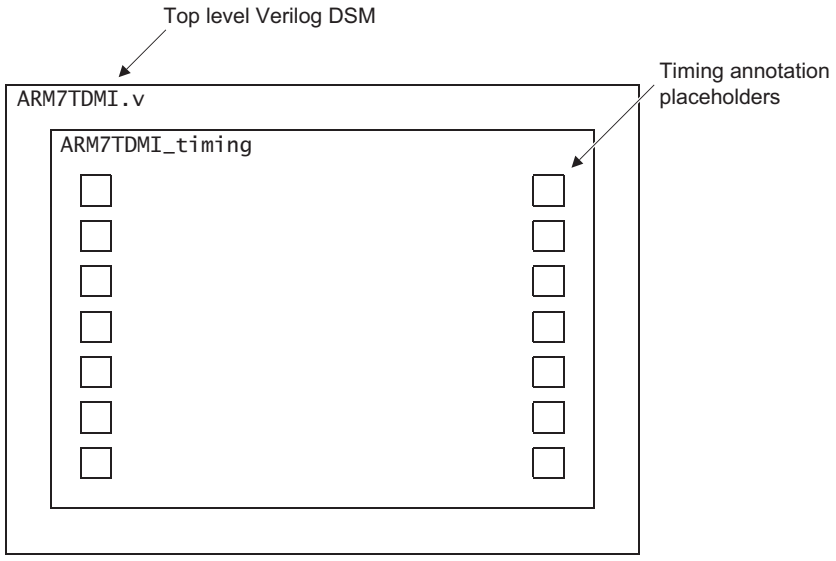


Figure 9-4 Timing annotation hierarchy

9.4.2 SDF remap

This section describes the SDF remap process. Figure 9-5 on page 9-10 shows this process in overview.

Copyright © 2005 ARM Limited. All rights reserved.
Confidential

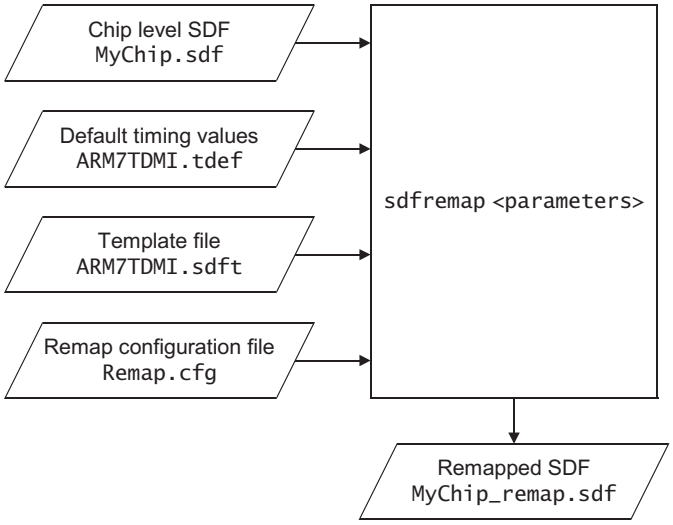


Figure 9-5 SDF remap process

The process involves a set of parameters:

Chip level SDF

The chip level SDF file is the SDF output from your timing analysis tool, for example `MyChip.sdf`.

Default timing values

The default timing values file, `ARM7TDMI.tdef`, is supplied with the DSM. The file provides the default timing parameters for any values that are not present in the SDF.

Template file

The template file, `ARM7TDMI.sdft`, is supplied with the DSM. The file is a template that the remap tool uses to specify the format of the resultant SDF to permit clean annotation.

Remap configuration file

This is the remap configuration file that you must create. It is recommended that you name this file `Remap.cfg`. The file must have these entries:

```
<celleq>
“ARM7TDMI”
“ARM7TDMI_timing”
<pathtrails>
```


Copyright © 2005 ARM Limited. All rights reserved.
Confidential

The first three lines tell the remap tool that ARM7TDMI and ARM7TDMI_timing are equivalent cells and that it must look at them.

The <pathtrails> line tells the tool to automatically add the change of hierarchy to the remapped SDF to compensate for the timing shell.

Remapped SDF

The remapped SDF file is the output of the SDF remap process, for example MyChip_remap.sdf. You use this output to annotate timing information on to the DSM.

How to use the sdfremap tool

To remap the SDF, see Figure 9-5 on page 9-10, use the example flow:

1. Find the tools, template file, and default timing values.

   The sdfremap tool is in the SDF_TOOLS directory of the DSM release.

   The template file, ARM7TDMI.sdft, is in the ARM7TDMI directory of the DSM release.

   The default timing view, ARM7TDMI.tdef, is in the ARM7TDMI directory of the DSM release.

2. Copy the following files to a single run directory:
   - sdfremap (binary)
   - ARM7TDMI.sdft
   - ARM7TDMI.tdef
   - MyChip.sdf

   Note

   The tool requires all these files in the same directory.

3. Create the remap configuration file, Remap.cfg. This file must be in your run directory.

4. Run the remap tool:

   ```
   sdfremap -v -p Remap.cfg -cell ARM7TDMI MyChip.sdf MyChip_remap.sdf
   ```

Annotating the SDF

When you have generated the remapped SDF, for example in MyChip_remap.sdf, you can now annotate the timing cleanly on to your design.

Copyright © 2005 ARM Limited. All rights reserved.
Confidential

Copyright © 2005 ARM Limited. All rights reserved.
Confidential

Chapter 10
Sign Off

This chapter describes the recommendations for sign-off checks for an SoC design including the macrocell. It contains the following sections:

Copyright © 2005 ARM Limited. All rights reserved.
Confidential

10.1 About sign off

This section summarizes specific sign off issues related to a design incorporating the macrocell:

- *Functional verification*
- *DFT verification*
- *Timing sign off*
- *Physical verification*.

Note

It is strongly recommended that you do all of these sign-off checks.

10.1.1 Functional verification

Before sign off you must make sure that all the configuration signals in your design are connected correctly and that the clocks are correctly configured. See Chapter 3 *Functional Integration Guidelines* and the relevant technical reference manuals for both the processor core and associated support devices.

10.1.2 DFT verification

You must use the DSM, provided with the macrocell, to prove the test functionality. You must simulate all of the test patterns created for your chosen DFT methodology.

10.1.3 Timing sign off

It is recommended that you use STA, with the supplied timing models, as your primary method of timing sign off. You can add dynamic simulation, using back-annotated SDF, to improve confidence in timing verification.

10.1.4 Physical verification

You must make sure that:

- the macrocell satisfies the requirements for IR drops
- all macrocell inputs and outputs are free of antenna rule violations
- all LVS and DRC checks are free of errors.

Copyright © 2005 ARM Limited. All rights reserved.
Confidential

Glossary

This glossary describes some of the terms used in technical documents from ARM Limited.

Advanced High-performance Bus (AHB)
A bus protocol with a fixed pipeline between address/control and data phases. It only supports a subset of the functionality provided by the AMBA AXI protocol. The full AMBA AHB protocol specification includes a number of features that are not commonly required for master and slave IP developments and ARM Limited recommends only a subset of the protocol is usually used. This subset is defined as the AMBA AHB-Lite protocol.

See also Advanced Microcontroller Bus Architecture and AHB-Lite.

Advanced Microcontroller Bus Architecture (AMBA)
A family of protocol specifications that describe a strategy for the interconnect. AMBA is the ARM open standard for on-chip buses. It is an on-chip bus specification that details a strategy for the interconnection and management of functional blocks that make up a *System-on-Chip* (SoC). It aids in the development of embedded processors with one or more CPUs or signal processors and multiple peripherals. AMBA complements a reusable design methodology by defining a common backbone for SoC modules.

Copyright © 2005 ARM Limited. All rights reserved.
Confidential

Advanced Peripheral Bus (APB)
A simpler bus protocol than AXI and AHB. It is designed for use with ancillary or general-purpose peripherals such as timers, interrupt controllers, UARTs, and I/O ports. Connection to the main system bus is through a system-to-peripheral bus bridge that helps to reduce system power consumption.

AHB *See* Advanced High-performance Bus.

AHB-Lite A subset of the full AMBA AHB protocol specification. It provides all of the basic functions required by the majority of AMBA AHB slave and master designs, particularly when used with a multi-layer AMBA interconnect. In most cases, the extra facilities provided by a full AMBA AHB interface are implemented more efficiently by using an AMBA AXI protocol interface.

AMBA *See* Advanced Microcontroller Bus Architecture.

APB *See* Advanced Peripheral Bus.

Application Specific Integrated Circuit (ASIC)
An integrated circuit that has been designed to perform a specific application function. It can be custom-built or mass-produced.

ASIC *See* Application Specific Integrated Circuit.

ATPG *See* Automatic Test Pattern Generation.

Automatic Test Pattern Generation (ATPG)
The process of automatically generating manufacturing test vectors for an ASIC design, using a specialized software tool.

Back-annotation The process of applying timing characteristics from the implementation process onto a model.

Boundary scan chain
A boundary scan chain is made up of serially-connected devices that implement boundary scan technology using a standard JTAG TAP interface. Each device contains at least one TAP controller containing shift registers that form the chain connected between **TDI** and **TDO**, through which test data is shifted. Processors can contain several shift registers to enable you to access selected parts of the device.

Byte An 8-bit data item.

Cold reset Also known as power-on reset. Starting the processor by turning power on. Turning power off and then back on again clears main memory and many internal settings. Some program failures can lock up the processor and require a cold reset to enable the system to be used again. In other cases, only a warm reset is required.

See also Warm reset.

Copyright © 2005 ARM Limited. All rights reserved.
Confidential

Condensed Reference Format (CRF)
An ARM proprietary file format for specifying test vectors.

Coprocessor A processor that supplements the main processor. It carries out additional functions that the main processor cannot perform. Usually used for floating-point math calculations, signal processing, or memory management.

Core A core is that part of a processor that contains the ALU, the datapath, the general-purpose registers, the Program Counter, and the instruction decode and control circuitry.

Core reset *See* Warm reset.

CRF *See* Condensed Reference Format.

DBGTAP *See* Debug Test Access Port.

Debug Test Access Port (DBGTAP)
The collection of four mandatory and one optional terminals that form the input/output and control interface to a JTAG boundary-scan architecture. The mandatory terminals are **DBGTDI**, **DBGTDO**, **DBGTMS**, and **TCK**. The optional terminal is **TRST**. This signal is mandatory in ARM cores because it is used to reset the debug logic.

Design Simulation Model (DSM)
A functional simulation model of the device that is derived from the *Register Transfer Level* (RTL) but that does not reveal its internal structure. The DSM does not model any features added during synthesis such as internal scan chains. The DSM provides higher speed for functional simulation than that of the Sign-Off Model (SOM).

DSM *See* Design Simulation Model.

EmbeddedICE logic An on-chip logic block that provides TAP-based debug support for ARM processor cores. It is accessed through the TAP controller on the ARM core using the JTAG interface.

EmbeddedICE-RT The JTAG-based hardware provided by debuggable ARM processors to aid debugging in real-time.

Embedded Trace Buffer
The ETB provides on-chip storage of trace data using a configurable sized RAM.

Embedded Trace Macrocell (ETM)
A hardware macrocell that, when connected to a processor core, outputs instruction and data trace information on a trace port. The ETM provides processor driven trace through a trace port compliant to the ATB protocol.

ETB *See* Embedded Trace Buffer.

ETM See *Embedded Trace Macrocell*.

Copyright © 2005 ARM Limited. All rights reserved.
Confidential

Host A computer that provides data and other services to another computer. Especially, a computer providing debugging services to a target being debugged.

Internal scan chain A series of registers connected together to form a path through a device, used during production testing to import test patterns into internal nodes of the device and export the resulting values.

Joint Test Action Group (JTAG)
The name of the organization that developed standard IEEE 1149.1. This standard defines a boundary-scan architecture used for in-circuit testing of integrated circuit devices. It is commonly known by the initials JTAG.

JTAG *See* Joint Test Action Group.

Macrocell A complex logic block with a defined interface and behavior. A typical VLSI system comprises several macrocells (such as a processor, an ETM, and a memory block) plus application-specific logic.

Microprocessor *See* Processor.

Multi-ICE A JTAG-based tool for debugging embedded systems.

Multi-layer An interconnect scheme similar to a cross-bar switch. Each master on the interconnect has a direct link to each slave, The link is not shared with other masters. This enables each master to process transfers in parallel with other masters. Contention only occurs in a multi-layer interconnect at a payload destination, typically the slave.

Multi-master AHB Typically a shared, not multi-layer, AHB interconnect scheme. More than one master connects to a single AMBA AHB link. In this case, the bus is implemented with a set of full AMBA AHB master interfaces. Masters that use the AMBA AHB-Lite protocol must connect through a wrapper to supply full AMBA AHB master signals to support multi-master operation.

Power-on reset *See* Cold reset.

Processor A processor is the circuitry in a computer system required to process data using the computer instructions. It is an abbreviation of microprocessor. A clock source, power supplies, and main memory are also required to create a minimum complete working computer system.

RealView ICE A system for debugging embedded processor cores using a JTAG interface.

Scan chain A scan chain is made up of serially-connected devices that implement boundary scan technology using a standard JTAG TAP interface. Each device contains at least one TAP controller containing shift registers that form the chain connected between **TDI** and **TDO**, through which test data is shifted. Processors can contain several shift registers to enable you to access selected parts of the device.

Copyright © 2005 ARM Limited. All rights reserved.
Confidential

SDF *See* Standard Delay Format.

Standard Delay Format (SDF)
The format of a file that contains timing information to the level of individual bits of buses and is used in SDF back-annotation. An SDF file can be generated in a number of ways, but most commonly from a delay calculator.

TAP *See* Test access port.

Test Access Port (TAP)
The collection of four mandatory and one optional terminals that form the input/output and control interface to a JTAG boundary-scan architecture. The mandatory terminals are **TDI**, **TDO**, **TMS**, and **TCK**. The optional terminal is **TRST**. This signal is mandatory in ARM cores because it is used to reset the debug logic.

Trace port A port on a device, such as a processor or ASIC, used to output trace information.

Trace Port Analyzer (TPA)
A hardware device that captures trace information output on a trace port. This can be a low-cost product designed specifically for trace acquisition, or a logic analyzer.

Warm reset Also known as a core reset. Initializes the majority of the processor excluding the debug controller and debug logic. This type of reset is useful if you are using the debugging features of a processor.

Word A 32-bit data item.

Copyright © 2005 ARM Limited. All rights reserved.
Confidential

Copyright © 2005 ARM Limited. All rights reserved.
Confidential